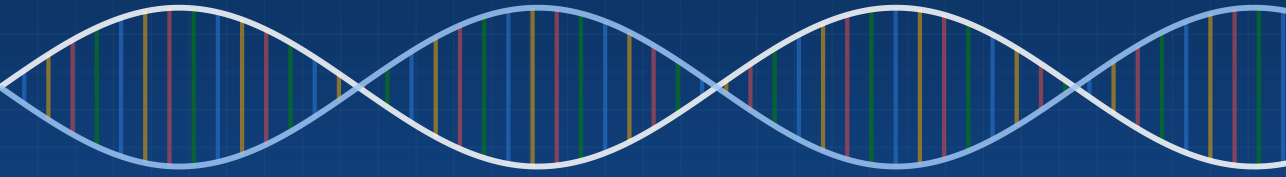


Ensamblaje y anotación de genomas



Abandonamos el enfoque clásico de «solapamiento–disposición–consenso» en favor de un nuevo algoritmo, EULER, que por primera vez resuelve el «problema de las repeticiones», con veinte años de antigüedad, en el ensamblaje de fragmentos.

— Pevzner et al. (2001)

● Al terminar este capítulo usted podrá

- Contar k -mers, construir e interpretar un espectro de k -mers, y usarlo para estimar sin referencia el tamaño, la heterocigosidad y la tasa de error de un genoma.
- Formular el ensamblaje como un problema de grafos, distinguir el paradigma de solapamiento (camino hamiltoniano) del de De Bruijn (camino euleriano) y reconocer puntas, burbujas y repeticiones en un grafo real.
- Ejecutar un ensamblaje con SPAdes y evaluarlo críticamente con N50, NG50, QUAST, BUSCO y métricas basadas en k -mers, sin dejarse engañar por la contigüidad aparente.

- Explicar cómo se predicen genes en procariotas y eucariotas (modelos de Márkov y modelos ocultos de Márkov), por qué se enmascaran las repeticiones y cómo integran la evidencia herramientas como Prokka, Bakta, AUGUSTUS y BRAKER.

Imagine que alguien toma cien ejemplares del mismo periódico, los pasa por una trituradora y le entrega a usted la montaña de tiras de papel resultante, pidiéndole que reconstruya la edición original. No hay foto de la portada, las tiras son cortas, algunas tienen manchas de tinta que cambian una letra por otra, y el periódico, como todos, repite frases enteras: los titulares de la sección de deportes, los anuncios, la tabla del clima. Ese es, sin exageración, el problema del *ensamblaje de genomas*. La secuenciación nos entrega millones de lecturas de un genoma que nunca vimos completo (??), y la tarea computacional consiste en deducir de qué texto largo proceden.

La tarea no termina ahí. Un genoma ensamblado es un texto de millones de letras sin puntuación ni espacios; para que sea útil hay que *anotarlo*: decir dónde empiezan y terminan los genes, qué proteínas codifican, dónde están los ARN ribosómicos y de transferencia, qué regiones son repeticiones de elementos móviles. Ensamblaje y anotación son las dos mitades del trabajo que convierte un archivo FASTQ en un recurso biológico.

El capítulo sigue ese orden. Empezaremos por la unidad mínima de casi todos los algoritmos modernos, el k -mer, y veremos que un simple histograma de sus frecuencias revela el tamaño del genoma, su heterocigosidad y la calidad de los datos antes de ensamblar nada (sección 8.1). Después convertiremos los k -mers en un grafo y descubriremos que un cambio de representación transforma un problema intratable en uno lineal (sección 8.2). Con esas ideas entenderemos qué hace un ensamblador real como SPAdes y, sobre todo, cómo juzgar su resultado (sección 8.3). Terminaremos leyendo el genoma ensamblado para encontrar sus genes (sección 8.4).

8.1 k -mers y espectros de k -mers



Medir un libro sin leerlo

Suponga que le entregan cuarenta fotocopias de un libro, cada una cortada en frases sueltas, todo mezclado en una caja. No sabe cuántas páginas tiene el libro, pero puede contar cuántas veces aparece cada frase de, digamos, seis palabras. La mayoría aparecerá unas cuarenta veces, una por fotocopia; si cuenta el total de frases de la caja y lo divide entre cuarenta, obtendrá la longitud del libro sin haberlo leído. Además, las frases que aparecen una sola vez delatan erratas de la fotopiadora; las que aparecen ochenta veces, estribillos que el autor repitió; y si el libro es en realidad *dos* ediciones ligeramente distintas mezcladas, habrá un grupo de frases que aparece veinte veces, las que difieren entre ediciones. Todo eso está escrito en un solo histograma.

8.1.1 Qué es un k -mer

La lectura de un secuenciador es demasiado larga y demasiado ruidosa para compararla entera con todas las demás, y demasiado corta para situarla sola en el genoma. Casi todos los algoritmos de ensamblaje, y muchos de mapeo, clasificación metagenómica y detección de variantes, trabajan con una unidad intermedia: las subcadenas de longitud fija.

Definición 8.1. (k -mer y k -mer canónico)

Un k -mer es una subcadena de longitud k de una secuencia. Una lectura s de longitud L contiene $L - k + 1$ k -mers, $s_{i..i+k-1}$ para $i = 1, \dots, L - k + 1$. Como no sabemos de qué hebra del ADN procede cada lectura, se identifica cada k -mer x con su reverso complementario $\bar{x}$ y se representa la pareja por su k -mer canónico

$$\kappa(x) = \min\{x, \bar{x}\}, \quad (8.1)$$

donde el mínimo es en orden lexicográfico ($A < C < G < T$).

DONDE	
k	Longitud de las palabras que se cuentan (típicamente entre 21 y 127 para lecturas cortas).
L	Longitud de la lectura en pares de bases.
$\bar{x}$	Reverso complementario de x : se invierte el orden y se cambia A $\leftrightarrow$ T, C $\leftrightarrow$ G.
$\kappa(x)$	Representante canónico: la misma clave para las dos hebras.

Hay 4^k k -mers posibles. Con $k = 21$ son unos $4,4 \times 10^{12}$, más de mil veces el número de posiciones del genoma humano; un k -mer tomado al azar del espacio posible casi nunca estará en un genoma concreto. Esa escasez es precisamente lo que los hace útiles como “huellas”: si un k -mer largo aparece en una lectura y en un genoma, es muy probable que la lectura venga de ahí.

Un detalle con consecuencias prácticas: casi todas las herramientas usan k impar. La razón es que un k -mer de longitud impar nunca es su propio reverso complementario, porque la base central tendría que ser igual a su complemento, lo que es imposible. Con k par existen palíndromos como **GAATTC** (el sitio de *EcoRI*), para los que $x = \bar{x}$ y la hebra queda ambigua; con k impar, cada k -mer canónico corresponde exactamente a dos k -mers orientados distintos.

Por eficiencia, los k -mers no se guardan como texto sino como enteros: cada base ocupa 2 bits (A= 00, C= 01, G= 10, T= 11), de modo que un k -mer con $k \leq 32$ cabe en una palabra de 64 bits. El paso de un k -mer al siguiente dentro de la lectura se hace con un desplazamiento de bits y una máscara, en tiempo constante, y el orden lexicográfico de la ecuación (8.1) coincide con el orden numérico de los enteros.

8.1.2 Cobertura de k -mers

En el ?? definimos la cobertura de un experimento como $c = NL/G$, el número medio de lecturas que cubren una base. Para los k -mers la cuenta es ligeramente distinta, porque una lectura cubre un k -mer solo si lo contiene *entero*.

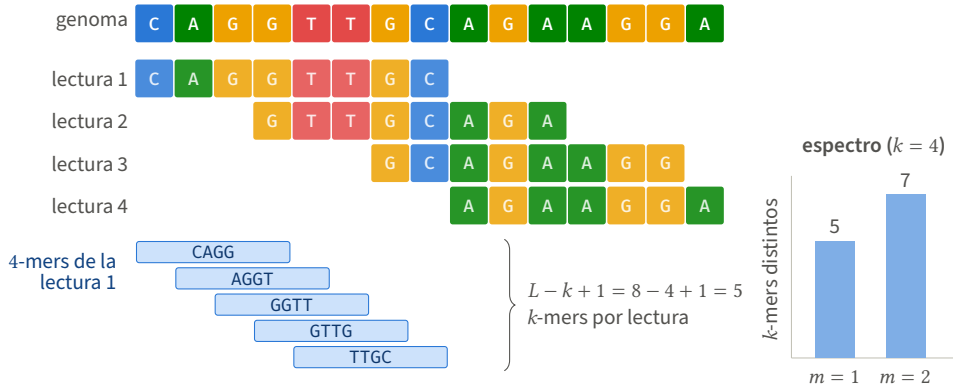


Figura 8.1. De las lecturas a los k -mers. Un genoma de 15 pb (arriba) se ha secuenciado con cuatro lecturas solapadas. Cada lectura se descompone en $L - k + 1$ palabras de longitud k ; aquí se muestran los cinco 4-mers de la primera lectura. Contando los 4-mers canónicos de las cuatro lecturas se obtienen 12 distintos: 5 aparecen una vez y 7 dos veces (derecha). Ese histograma, el *espectro de k -mers*, es el objeto central de esta sección. Observe que la información de *dónde* estaba cada k -mer se ha perdido: solo queda *cuántas* veces aparece.

Teorema 8.1. (Cobertura de k -mers)

Con N lecturas de longitud L tomadas al azar de un genoma de tamaño G y una tasa de error por base e independiente entre posiciones, el número medio de veces que se observa sin errores un k -mer del genoma es

$$\lambda_k = \underbrace{\frac{N(L - k + 1)}{G}}_{c_k = c \frac{L - k + 1}{L}} \cdot (1 - e)^k. \quad (8.2)$$

DONDE

- c Cobertura por bases, NL/G (??).
- c_k Cobertura de k -mers sin tener en cuenta los errores.
- e Probabilidad de que una base de la lectura sea errónea.
- $(1 - e)^k$ Probabilidad de que las k bases de una aparición del k -mer sean todas correctas.
- λ_k Cobertura efectiva de k -mers: la posición del pico principal del espectro.

Derivación. Una lectura contiene el k -mer que empieza en la posición i del genoma si la lectura empieza en alguna de las $L - k + 1$ posiciones $i - (L - k), \dots, i$. Con G posiciones de inicio equiprobables, cada lectura lo contiene con probabilidad $(L - k + 1)/G$, y el número esperado de lecturas que lo contienen es $N(L - k + 1)/G = c(L - k + 1)/L$. De esas apariciones, solo cuentan como el k -mer correcto las que no tienen ningún error en sus k bases, lo que ocurre con probabilidad $(1 - e)^k$. Las demás se convierten en k -mers *distintos*, casi siempre únicos. $\square$

La ecuación (8.2) tiene dos factores que crecen en direcciones opuestas con k , y los dos castigan los k grandes: $(L - k + 1)/L$ disminuye linealmente y $(1 - e)^k$ lo hace geométricamente. Con lecturas de 150 pb, $e = 0,5\%$ y $k = 21$, una cobertura de $20\times$ se convierte en $c_k = 17,3$ y en una cobertura efectiva $\lambda_k = 17,3 \times 0,900 = 15,6$. Uno de cada diez k -mers leídos contiene al menos un error.

8.1.3 El espectro de k -mers

Definición 8.2. (Espectro de k -mers)

Sea $n(x)$ el número de veces que el k -mer canónico x aparece en el conjunto de lecturas. El *espectro de k -mers* es la función

$$h(m) = \#\{x : n(x) = m\}, \quad m = 1, 2, 3, \dots \quad (8.3)$$

es decir, el número de k -mers *distintos* que aparecen exactamente m veces. Su “masa” $\sum_m m h(m)$ es el número total de k -mers leídos.

DONDE

$n(x)$	Multiplicidad del k -mer x : cuántas veces se leyó.
m	Multiplicidad (eje horizontal del espectro).
$h(m)$	Número de k -mers distintos con multiplicidad m (eje vertical).

Si el genoma fuera haploide, sin repeticiones y sin errores de secuenciación, cada uno de sus $\approx G$ k -mers se observaría un número de veces aproximadamente de Poisson con media λ_k (por el mismo argumento de Lander & Waterman, 1988, visto en el ??), y el espectro sería una única campana centrada en λ_k con área G . Los genomas reales añaden tres ingredientes, cada uno con una firma inconfundible:

- **Errores de secuenciación.** Cada error crea hasta k k -mers que no existen en el genoma y que casi nunca se repiten. Forman un pico enorme en $m = 1-3$ que decae muy deprisa.
- **Heterocigosidad.** En un diploide, los k -mers que cubren un sitio heterocigoto existen en una sola de las dos copias del cromosoma y se leen la mitad de veces que los demás. Forman un segundo pico a la mitad de la cobertura.
- **Repeticiones.** Un k -mer presente en r copias del genoma se lee r veces más. Forman picos (o colas) en múltiplos de la cobertura.

8.1.4 Un modelo para el espectro de un diploide

Formalicemos la heterocigosidad, que es la parte más útil del modelo. Sea h la tasa de heterocigosidad: la probabilidad de que una posición del genoma difiera entre los dos haplotipos. Sea λ la cobertura efectiva de k -mers *por haplotipo*, de modo que un k -mer presente en los dos haplotipos se lee en promedio 2λ veces.

Teorema 8.2. (Espectro de un genoma diploide)

Un k -mer del genoma cubre al menos un sitio heterocigoto con probabilidad

$$\alpha = 1 - (1 - h)^k. \quad (8.4)$$

Si el genoma haploide tiene G posiciones, hay en promedio $G(1 - \alpha)$ k -mers homocigotos, con multiplicidad media 2λ , y $2G\alpha$ k -mers heterocigotos (uno por haplotipo), con multiplicidad media λ . Ignorando repeticiones, el espectro es la mezcla

$$h(m) \approx h_{err}(m) + 2G\alpha f(m; \lambda) + G(1 - \alpha) f(m; 2\lambda), \quad (8.5)$$

donde $f(\cdot; \mu)$ es una distribución de conteo de media μ (Poisson o, más realista, binomial negativa).

DONDE

h	Tasa de heterocigosidad: fracción de sitios que difieren entre los dos haplotipos.
α	Fracción de posiciones cuyo k -mer contiene al menos un sitio heterocigoto.
λ	Cobertura efectiva de k -mers de un haplotipo (el pico “1×”).
$h_{\text{err}}(m)$	Contribución de los k -mers erróneos, concentrada en m pequeños.
$f(m; \mu)$	Probabilidad de observar m copias cuando la media es μ .

La ecuación (8.4) explica una observación que sorprende a muchos: una heterocigosidad modesta produce un pico heterocigoto enorme. Con $h = 1\%$ y $k = 21$, $\alpha = 1 - 0,99^{21} = 0,19$: casi uno de cada cinco k -mers es heterocigoto, y como cada posición heterocigota aporta *dos* k -mers distintos (uno por alelo), el pico de λ contiene la mitad de k -mers que el de 2λ . El efecto crece con k , porque una palabra más larga tiene más oportunidades de tocar un sitio variable.

Estimación del tamaño del genoma. El modelo contiene un resultado de gran belleza práctica. Calculemos la masa total de los k -mers verdaderos (los que no son errores):

$$\underbrace{G(1 - \alpha) \cdot 2\lambda}_{\text{homocigotos}} + \underbrace{2G\alpha \cdot \lambda}_{\text{heterocigotos}} = 2G\lambda \implies \hat{G} = \frac{\sum_{m \geq v} m h(m)}{2\lambda}. \quad (8.6)$$

La heterocigosidad α se cancela. Basta sumar el espectro por encima del valle v que lo separa de los errores y dividir por la multiplicidad del pico homocigoto, 2λ . Las repeticiones tampoco estorban: un k -mer con r copias aporta r veces más masa, exactamente lo que le corresponde. Para un organismo haploide, el denominador es simplemente la posición del único pico, λ .

Estimación de la heterocigosidad. Si ajustamos la ecuación (8.5) y obtenemos los tamaños de los dos componentes, $N_{\text{het}} \approx 2G\alpha$ y $N_{\text{hom}} \approx G(1 - \alpha)$, despejamos

$$\hat{\alpha} = \frac{N_{\text{het}}}{N_{\text{het}} + 2N_{\text{hom}}}, \quad \hat{h} = 1 - (1 - \hat{\alpha})^{1/k}. \quad (8.7)$$

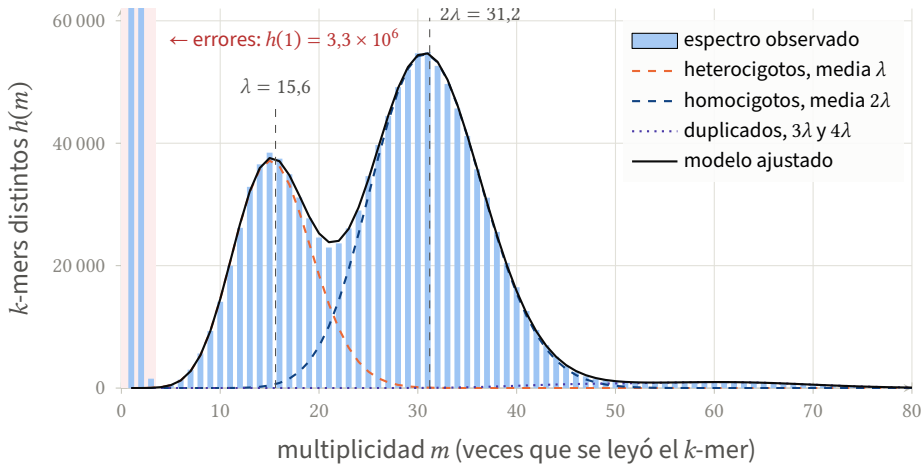


Figura 8.2. Espectro de 21-mers de un genoma diploide simulado de 1 Mb con heterocigosidad del 1 %, 15 duplicaciones segmentarias de 2 kb, lecturas de 150 pb a 20× por haplotipo y 0,5 % de errores. La zona roja ($m \leq 3$) contiene los k -mers erróneos: son el 74 % de los k -mers *distintos*, aunque solo el 10 % de los leídos. A la derecha del valle aparecen el pico heterocigoto (naranja), en λ , y el homocigoto (azul), en 2λ , más una pequeña cola de k -mers duplicados. El pico heterocigoto tiene alrededor de la mitad de k -mers que el homocigoto, como predice la ecuación (8.4) con $\alpha \approx 0,19$. El ajuste de la ecuación (8.5) recupera $\hat{G} = 1\,000\,409$ pb y $\hat{h} = 1,02$ %. Generado con `figuras/cap08/generar.py`.

DONDE

v	Multiplicidad del valle entre el pico de errores y el primer pico genómico.
$\hat{G}$	Tamaño estimado del genoma haploide.
$N_{\text{het}}, N_{\text{hom}}$	Número de k -mers distintos asignados a cada componente del ajuste.

Este es, en esencia, el modelo de GenomeScope (Vurture et al., 2017), que ajusta por mínimos cuadrados no lineales una mezcla de binomiales negativas en λ , 2λ , 3λ y 4λ (esta última para capturar las repeticiones de dos copias) y devuelve el tamaño del genoma, la heterocigosidad, la fracción repetitiva y la tasa de error a partir de lecturas sin procesar. Sus autores lo validaron con 324 conjuntos de datos simulados y 16 reales. GenomeScope 2.0 generalizó el modelo a poliploides mediante un razonamiento combinatorio sobre cuántos haplotipos comparten cada k -mer, y lo acompañó de Smudgeplot, que estima la ploidía a partir de pares de k -mers heterocigotos (Ranallo-Benavidez et al., 2020).

Ejemplo 8.1. (Perfilar un genoma antes de ensamblarlo)

Tomamos el conjunto simulado de la figura 8.2. El conteo produce 34 666 580 21-mers leídos, de los cuales 4 535 463 son distintos y 3 297 024 aparecen una sola vez.

1. *Valle.* El mínimo local entre el pico de errores y el primer pico genómico está en $v = 4$ ($h(4) = 166$).
2. *Cobertura.* El ajuste da $\lambda = 15,60$. La ecuación (8.2) predice $20 \times \frac{130}{150} \times 0,995^{21} = 17,33 \times 0,900 = 15,60$: la coincidencia confirma que el desplazamiento del pico respecto de c_k se debe a los errores.
3. *Tamaño.* La masa por encima del valle es $\sum_{m \geq 4} m h(m) = 31\,213\,016$, luego

$$\hat{G} = \frac{31\,213\,016}{2 \times 15,60} = 1\,000\,409 \text{ pb},$$

a menos de un 0,05 % del valor real (10^6 pb).

4. *Heterocigosidad.* El ajuste asigna $N_{\text{het}} = 367\,251$ y $N_{\text{hom}} = 765\,306$ k -mers distintos a los dos picos. Con la ecuación (8.7): $\hat{\alpha} = 367\,251 / (367\,251 + 2 \times 765\,306) = 0,1935$ y $\hat{h} = 1 - (1 - 0,1935)^{1/21} = 1,02\%$, frente al 1,00 % simulado.

Sin alinear una sola lectura contra nada, sabemos el tamaño del genoma, que es diploide, cuán heterocigoto es y que la cobertura es suficiente. Con esa información se elige el ensamblador y sus parámetros.

⚠ Confundir los picos duplica el genoma

Si la heterocigosidad es alta o la cobertura baja, el pico heterocigoto puede ser *más alto* que el homocigoto. Un análisis ingenuo que tome el pico más alto como “la cobertura” usará λ en lugar de 2λ en la ecuación (8.6) y estimará un genoma del *doble* de tamaño. Otros motivos de espectros engañosos son la contaminación (un tercer pico ajeno, a otra cobertura), la amplificación del genoma completo (picos anchos y deformados) y las muestras de varios individuos mezclados. Mire siempre el espectro completo, no solo los números que devuelve el programa.

8.1.5 Contar miles de millones de *k*-mers

Contar *k*-mers parece trivial, y en principio lo es: basta un diccionario. El problema es la escala. Un genoma humano a 30× con lecturas de 150 pb contiene unos 8×10^{10} 31-mers leídos, y el número de *k*-mers distintos, inflado por los errores, puede superar los 5×10^9 . Un diccionario de Python necesitaría cientos de gigabytes.

Marçais y Kingsford (2011) resolvieron el problema con Jellyfish, un contador basado en una tabla *hash* de direccionamiento abierto cuyos incrementos se hacen con instrucciones atómicas del procesador (*compare-and-swap*). Así, muchos hilos pueden insertar en la misma tabla sin bloquearla (*lock-free*), y el conteo escala casi linealmente con el número de núcleos. La versión original admitía *k*-mers de hasta 31 bases, la longitud que cabe en 62 bits. KMC 3 (Kokot et al., 2017) adoptó una estrategia complementaria: reparte los *k*-mers en particiones en disco y las cuenta después por lotes, de modo que la memoria necesaria es una fracción de la de una tabla completa, e incluye utilidades para operar con los conjuntos resultantes (intersecciones, diferencias, filtros por multiplicidad).

```
espectro.py

from collections import Counter

COMP = str.maketrans("ACGT", "TGCA")

def canonico(kmer: str) -> str:
    """El menor entre el k-mer y su reverso complementario."""
    return min(kmer, kmer.translate(COMP)[::-1])

def contar_kmers(lecturas, k=21):
    c = Counter()
    for s in lecturas:
        for i in range(len(s) - k + 1):
            km = s[i:i + k]
            if "N" not in km:
                c[canonico(km)] += 1
    return c

def espectro(conteos):
    """h[m] = número de k-mers distintos vistos m veces."""
    return Counter(conteos.values())

lect = ["CAGGTTGC", "GTTGCAGA", "GCAGAAGG", "AGAAGGA"]
print(sorted(espectro(contar_kmers(lect, k=4)).items()))
```

```
# [(1, 5), (2, 7)] <- el espectro de la figura 8.1
```

Para datos reales se usan las herramientas especializadas. El flujo típico es contar, construir el histograma y ajustarlo:

Perfil del genoma con Jellyfish (o KMC) y GenomeScope

```
# 1) contar 21-mers canónicos (-C) con 8 hilos
jellyfish count -C -m 21 -s 1G -t 8 -o k21.jf \
  <(zcat lect_R1.fq.gz) <(zcat lect_R2.fq.gz)
jellyfish histo -t 8 k21.jf > k21.histo

# alternativa con KMC: contar e histograma
kmc -k21 -t8 -ci1 -cs100000 @archivos.txt k21 tmp/
kmc_tools transform k21 histogram k21.histo -cx100000

# 2) ajustar el modelo (diploide: -p 2)
genomescope2 -i k21.histo -o perfil/ -k 21 -p 2
```

8.1.6 Elegir k

La elección de k es un compromiso entre dos exigencias opuestas. Por un lado, k debe ser lo bastante grande para que los k -mers sean *únicos* en el genoma: si un k -mer aparece en muchos lugares por puro azar, deja de identificar una posición. Por otro, debe ser lo bastante pequeño para que los k -mers tengan cobertura suficiente y no se vean arruinados por los errores (ecuación (8.2)).

Cuantifiquemos la primera exigencia con un modelo nulo: un genoma aleatorio de G bases equiprobables. Un k -mer dado puede reaparecer en cualquiera de las $\approx 2G$ posiciones de las dos hebras, cada una con probabilidad 4^{-k} . El número de apariciones casuales es aproximadamente de Poisson con media $2G/4^k$, y la probabilidad de que haya al menos una es

$$p_{\text{azar}}(k) = 1 - \exp\left(-\frac{2G}{4^k}\right), \quad p_{\text{azar}} \ll 1 \iff k \gg \log_4(2G). \quad (8.8)$$

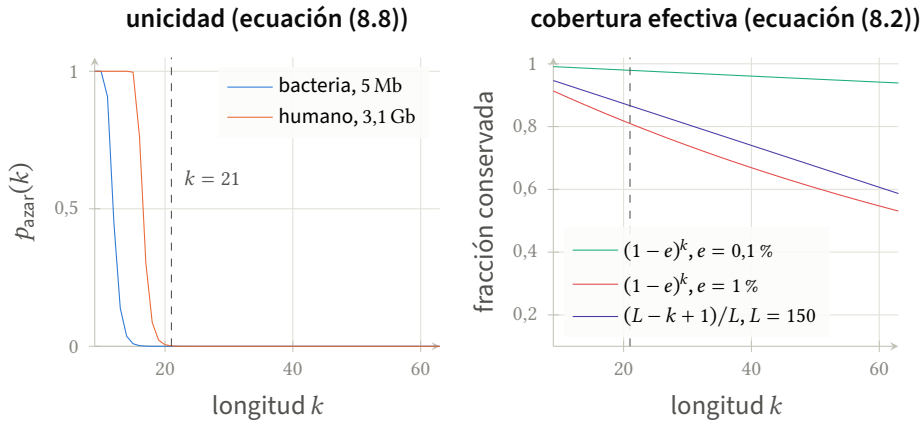


Figura 8.3. El compromiso en la elección de k . **Izquierda:** probabilidad de que un k -mer reaparezca por azar en un genoma aleatorio; la transición ocurre cerca de $\log_4(2G)$, unas cuatro bases más tarde en el genoma humano que en una bacteria. **Derecha:** los dos factores de la ecuación (8.2) que reducen la cobertura efectiva al crecer k . Con errores del 1 %, un 31-mer se conserva intacto solo el 73 % de las veces. Un k pequeño confunde posiciones; uno grande diluye la señal. Los ensambladores modernos resuelven el dilema usando varios valores de k a la vez (sección 8.3).

DONDE

$p_{\text{azar}}(k)$	Probabilidad de que un k -mer del genoma aparezca otra vez por azar (no por una repetición biológica).
$2G/4^k$	Número esperado de coincidencias casuales en las dos hebras.
$\log_4(2G)$	Longitud a partir de la cual los k -mers aleatorios empiezan a ser únicos.

Para una bacteria de 5 Mb, $\log_4 G \approx 11,1$; para el genoma humano, $\approx 15,8$. Con $k = 17$, un k -mer bacteriano tiene una probabilidad de coincidencia casual de 6×10^{-4} , pero uno humano, del 30 %. Con $k = 21$ ambas son despreciables (2×10^{-6} y $1,4 \times 10^{-3}$), y por eso $k = 21$ es un valor habitual para perfilar genomas. La figura 8.3 muestra el compromiso completo. Por supuesto, los genomas reales no son aleatorios: sus repeticiones biológicas (transposones, familias génicas, duplicaciones) son mucho más largas que cualquier k razonable, y ningún k las resuelve. Lo retomaremos en la sección 8.2.

Los k -mers como regla de medir la calidad

El espectro no solo sirve *antes* del ensamblaje. Si contamos por separado los k -mers de las lecturas y los del ensamblaje terminado, cada k -mer del ensamblaje que no aparece en las lecturas señala casi con certeza un error de consenso. Merqury (Rhie et al., 2020) convierte esa idea en una estimación de la exactitud por base sin necesidad de referencia, y usa los espectros de los padres para medir si un ensamblaje separa correctamente los dos haplotipos. Lo usaremos en la sección 8.3.

El espectro de k -mers es una radiografía gratuita del genoma: el pico en 2λ da la cobertura, la masa bajo la curva dividida por 2λ da el tamaño, y el pico en λ delata la heterocigosidad.

8.2 Grafos de De Bruijn

 Colab · 8.2 · Grafos de De Bruijn

 repositorio del curso

El cartero y el inspector

Dos funcionarios municipales reciben el plano de un barrio. El cartero debe recorrer *cada calle* exactamente una vez; el inspector debe visitar *cada casa* exactamente una vez. Parecen tareas hermanas, pero no lo son. El cartero tiene una regla sencilla para saber si su ruta existe: basta con que en cada esquina entren tantas calles como salen, y cualquier estudiante puede construir el recorrido en un par de minutos. El inspector, en cambio, no dispone de ninguna regla general: para un barrio grande, ni el mejor ordenador del mundo garantiza encontrar su ruta en un tiempo razonable. Durante dos décadas, los ensambladores de genomas trabajaron como el inspector. El gran avance consistió en redibujar el plano para que el problema fuera el del cartero.

8.2.1 El ensamblaje como problema de grafos

Formulado de la manera más ingenua, el ensamblaje busca la *supercadena común más corta*: la cadena más breve que contiene todas las lecturas como subcadenas. Es un problema NP-difícil, y además está mal planteado desde el punto de vista

biológico, porque la supercadena más corta colapsa todas las copias de una repetición en una sola. Las formulaciones útiles se apoyan en grafos.

Solapamiento–disposición–consenso (OLC). El paradigma clásico construye un *grafo de solapamiento*: cada lectura es un nodo, y se traza una arista $u \rightarrow v$ cuando un sufijo de u coincide con un prefijo de v en al menos T bases. Reconstruir el genoma equivale a encontrar un camino que visite cada nodo una vez, un *camino hamiltoniano*. El paradigma tiene tres fases: calcular todos los solapamientos (en principio, $O(N^2)$ comparaciones de lecturas), decidir la disposición de las lecturas y calcular la secuencia de consenso. Encontrar un camino hamiltoniano es NP-completo, así que los ensambladores OLC recurren a heurísticas. Myers (2005) dio al paradigma una forma más limpia, el *grafo de cadenas (string graph)*, que elimina las aristas transitivas (si $u \rightarrow v \rightarrow w$ y $u \rightarrow w$, la arista $u \rightarrow w$ no añade información) y deja un grafo en el que muchas regiones del genoma aparecen como caminos simples.

De Bruijn. El enfoque alternativo cambia los papeles de nodos y aristas. En lugar de lecturas, los nodos son palabras cortas de longitud $k-1$, y cada k -mer leído es una arista entre su prefijo y su sufijo. Idury y Waterman (1995) propusieron representar las lecturas de esta forma, y Pevzner et al. (2001) mostraron que el ensamblaje se reduce entonces a una variante del problema de encontrar un *camino euleriano*, que recorre cada arista una vez y se resuelve en tiempo lineal.

Definición 8.3. (Grafo de De Bruijn de un conjunto de lecturas)

Dado un conjunto de lecturas y un entero k , el *grafo de De Bruijn* $G_k = (V, E)$ es el multigrafo dirigido en el que

- V es el conjunto de $(k-1)$ -mers distintos presentes en las lecturas;
- por cada k -mer $x = x_1 x_2 \dots x_k$ se añade una arista del nodo $x_1 \dots x_{k-1}$ (su prefijo) al nodo $x_2 \dots x_k$ (su sufijo).

Si un k -mer aparece varias veces en el genoma, se conserva una arista por aparición (o una arista con multiplicidad).

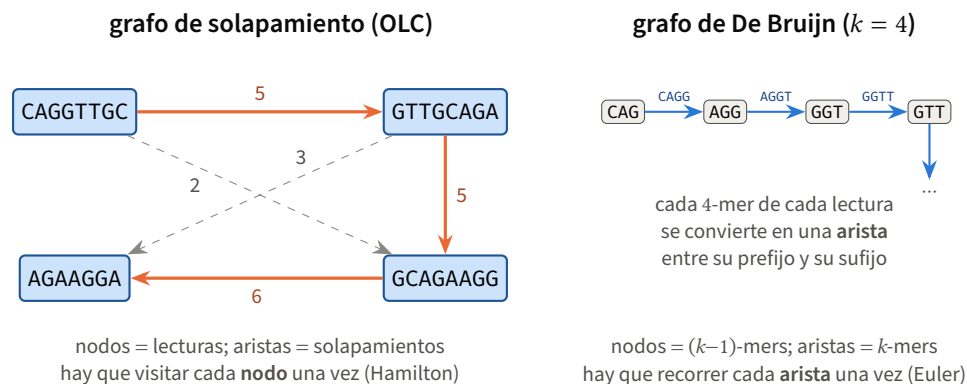


Figura 8.4. Las mismas cuatro lecturas de la figura 8.1 en los dos paradigmas. **Izquierda:** en el grafo de solapamiento cada lectura es un nodo y cada solapamiento sufijo–prefijo, una arista con su longitud; las aristas naranjas forman el camino hamiltoniano correcto, y las grises son solapamientos cortos espurios que un umbral $T \geq 4$ eliminaría. **Derecha:** en el grafo de De Bruijn las lecturas desaparecen como entidades; solo quedan sus k -mers, convertidos en aristas. El grafo completo se muestra en la figura 8.5.

8.2.2 El teorema de Euler

La razón por la que el cambio de representación importa es un teorema de 1736. Leonhard Euler lo formuló al estudiar si era posible pasear por Königsberg cruzando cada uno de sus siete puentes exactamente una vez; la versión para grafos dirigidos es la que necesitamos.

Teorema 8.3. (Caminos eulerianos en grafos dirigidos)

Sea $G = (V, E)$ un grafo dirigido conexo (ignorando la orientación de las aristas) y sean $d^-(v)$ y $d^+(v)$ el número de aristas que entran en y salen de v . Entonces:

1. G tiene un ciclo euleriano si y solo si $d^-(v) = d^+(v)$ para todo v (el grafo está balanceado).
2. G tiene un camino euleriano de s a t $s \neq t$ si y solo si

$$d^+(s) = d^-(s) + 1, \quad d^-(t) = d^+(t) + 1, \quad d^-(v) = d^+(v) \quad \forall v \notin \{s, t\}. \quad (8.9)$$

DONDE

$d^-(v), d^+(v)$	Grado de entrada y de salida del nodo v .
s, t	Nodos inicial y final del camino (en un genoma lineal, el primer y el último $(k - 1)$ -mer).

Por qué es cierto. La necesidad es casi evidente: cada vez que un recorrido entra en un nodo intermedio debe volver a salir, así que las aristas de entrada y salida se emparejan; solo el inicio tiene una salida “de más” y el final una entrada “de más”. La suficiencia es constructiva, y esa construcción es el algoritmo. Si añadimos una arista ficticia $t \rightarrow s$, el grafo queda balanceado y basta con encontrar un ciclo. Empiece en cualquier nodo y camine por aristas no usadas; como el grafo está balanceado, solo puede quedarse atascado en el nodo de partida, de modo que ha trazado un ciclo. Si quedan aristas sin usar, alguna sale de un nodo del ciclo (por conexidad); inicie allí un nuevo recorrido y empálmelo en el ciclo anterior. Repitiendo, se agotan las aristas. $\square$

Esta demostración es el *algoritmo de Hierholzer*, que se implementa con una pila y recorre cada arista una sola vez: tiempo $O(|E|)$. Para un genoma bacteriano con $|E| \approx 5 \times 10^6$ aristas, son milisegundos. Compárese con el camino hamiltoniano, para el que no se conoce ningún algoritmo polinómico. Compeau et al. (2011) explican con detalle por qué este cambio de perspectiva transformó el ensamblaje con lecturas cortas.

8.2.3 Un ejemplo completo

Tomemos el genoma de juguete $S = \text{CAGGTTGCAGAAGGA}$ (15 pb) y $k = 4$. Sus 12 4-mers son todos distintos, pero los 3-mers **CAG** y **AGG** aparecen dos veces cada uno: son una *repetición* de longitud $k - 1$. El grafo de De Bruijn (figura 8.5) tiene 11 nodos y 12 aristas. Los grados cumplen la ecuación (8.9): **CAG** tiene una salida de más (es s), **GGA** una entrada de más (es t), y **AGG**, con dos entradas y dos salidas, está balanceado.

La figura 8.5 muestra el fenómeno central del ensamblaje: el grafo admite *dos* caminos eulerianos,

$$\text{CAGGTTGCAGAAGGA} \quad \text{y} \quad \text{CAGAAGGTTGCAGGA},$$

que contienen exactamente los mismos 4-mers con la misma multiplicidad. Ningún algoritmo puede distinguirlos usando solo esos 4-mers; de hecho, la implementación de Hierholzer que damos más abajo devuelve la segunda, la incorrecta. Con $k = 5$ la repetición **CAG/AGG** queda dentro de nodos más largos y distintos, y el camino

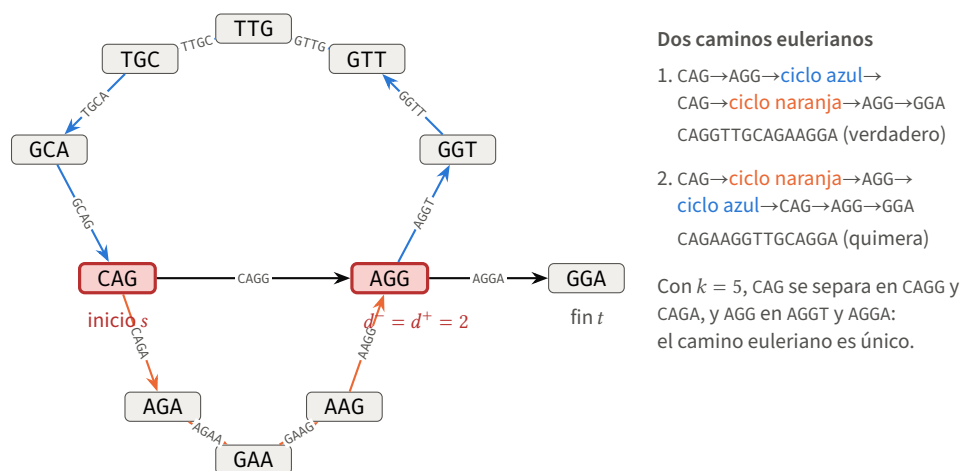


Figura 8.5. Grafo de De Bruijn con $k = 4$ del genoma CAGGTTGCAGAAGGA. Cada nodo es un 3-mer y cada arista, rotulada con su 4-mer, une el prefijo con el sufijo. Los nodos rojos, CAG y AGG, aparecen dos veces en el genoma: son una repetición de longitud $k - 1 = 3$ que crea dos ciclos (azul y naranja). Todo camino euleriano de CAG a GGA usa las 12 aristas, pero hay dos, que recorren los ciclos en órdenes distintos y deletrean dos secuencias igualmente compatibles con los datos. Solo una es el genoma. Los números se comprobaron con `figuras/cap08/generar.py`.

euleriano pasa a ser único. Esta es la forma precisa de la regla general: *si ningún $(k - 1)$ -mer se repite en el genoma, el grafo de De Bruijn es un camino simple y el ensamblaje es único*. Cada repetición más larga que $k - 1$ introduce, en cambio, ramificaciones cuyo orden de recorrido no está determinado por los datos.

🧪 ¿Cuántos caminos eulerianos hay?

El número de ciclos eulerianos de un grafo dirigido balanceado se puede contar exactamente con el teorema BEST (por de Bruijn, van Aardenne-Ehrenfest, Smith y Tutte): es el número de árboles de expansión orientados hacia un nodo raíz, multiplicado por $\prod_v (d^+(v) - 1)!$. Como el número de árboles de expansión puede ser exponencial, un genoma con muchas repeticiones admite una cantidad astronómica de reconstrucciones compatibles con los k -mers. Por eso los ensambladores no intentan elegir una: se detienen en las ramificaciones y entregan los tramos no ambiguos.

`de_bruijn.py`

```
from collections import defaultdict
```

```

def de_bruijn(lecturas, k):
    """Nodos: (k-1)-mers; una arista u->v por cada k-mer."""
    g = defaultdict(list)
    for s in lecturas:
        for i in range(len(s) - k + 1):
            g[s[i:i + k - 1]].append(s[i + 1:i + k])
    return g

def camino_euleriano(g, inicio):
    """Algoritmo de Hierholzer, O(|E|)."""
    g = {u: list(v) for u, v in g.items()} # copia consumible
    pila, camino = [inicio], []
    while pila:
        u = pila[-1]
        if g.get(u):
            # quedan aristas sin usar
            pila.append(g[u].pop())
        else:
            # callejón: fijar el nodo
            camino.append(pila.pop())
    camino.reverse()
    return camino[0] + "".join(v[-1] for v in camino[1:])

g = de_bruijn(["CAGGTTGCAGAAGGA"], k=4)
print(camino_euleriano(g, "CAG")) # CAGAAGGTTGCAGGA (quimera)
g = de_bruijn(["CAGGTTGCAGAAGGA"], k=5)
print(camino_euleriano(g, "CAGG")) # CAGGTTGCAGAAGGA (correcto)

```

8.2.4 Unitigs: lo que sí es seguro

Si no podemos elegir entre caminos, ¿qué podemos afirmar? Que los tramos *sin ramificaciones* son correctos. Un *unitig* es un camino máximo cuyos nodos internos tienen exactamente una entrada y una salida. En el ejemplo hay cuatro: **CAGG**, **AGGTTGCAG**, **CAGAAGG** y **AGGA**. Cualquier camino euleriano es una concatenación de estos unitigs (con solapamiento de $k - 1$ bases), y los ensambladores de lecturas cortas los entregan como *contigs* en lugar de arriesgarse con una elección. El *grafo compactado*, en el que cada unitig se reduce a un único nodo, tiene muchísimos menos nodos que el original y es la estructura con la que trabajan en la práctica.

8.2.5 El grafo real: puntas, burbujas y marañas

El ejemplo anterior era limpio: una lectura perfecta de cada k -mer. Con datos reales, el grafo se llena de estructuras parásitas (figura 8.6).

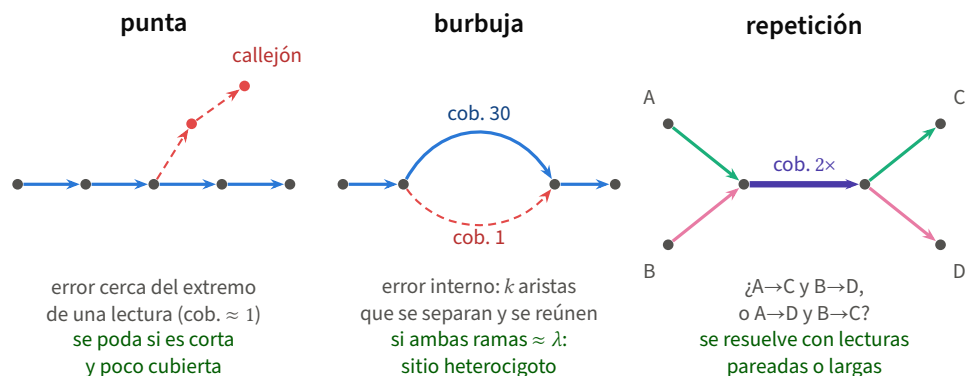


Figura 8.6. Las tres estructuras que complican un grafo de De Bruijn real (cada flecha representa un camino no ramificado). **Punta:** un error cerca del extremo de una lectura genera una rama corta sin salida. **Burbuja:** un error interior, o un sitio heterocigoto, crea dos caminos paralelos; la cobertura de cada rama distingue ambos casos, lo mismo que el espectro de la figura 8.2. **Repetición:** dos regiones distintas del genoma (A y B) comparten una secuencia más larga que k , que el grafo colapsa en un único tramo con el doble de cobertura; los datos del grafo no dicen si A continúa hacia C o hacia D.

- **Puntas (tips).** Un error cerca del extremo de una lectura crea un camino corto que se desprende del grafo y termina en un callejón sin salida, con cobertura muy baja.
- **Burbujas (bubbles).** Un error en el interior de una lectura crea un camino alternativo de k aristas que se separa y vuelve a unirse al camino principal. Un sitio heterocigoto produce exactamente la misma forma, pero con las dos ramas a cobertura parecida (cada una a λ , frente a 2λ del resto).
- **Marañas (tangles).** Una repetición más larga que k se colapsa en un solo camino por el que pasan varios recorridos del genoma; el tramo colapsado tiene una cobertura igual a la de una región única multiplicada por el número de copias.

¿Cuántas de estas estructuras hay? Cada error de secuenciación en el interior de una lectura genera hasta k k -mers inexistentes en el genoma. Con NL bases leídas y tasa de error e , el número de k -mers espurios es aproximadamente

$$\mathbb{E}[\#k\text{-mers erróneos}] \approx NLe^k. \quad (8.10)$$

DONDE

NL	Número total de bases leídas ($= cG$).
NLe	Número esperado de errores de secuenciación.
k	Cada error contamina hasta k k -mers consecutivos.

Para un genoma humano a $30\times$ ($NL = 9,3 \times 10^{10}$) con $e = 0,1\%$ y $k = 31$, la ecuación (8.10) da unos $2,9 \times 10^9$ k -mers espurios: casi tantos como los 3×10^9 verdaderos. El grafo “en bruto” es el doble de grande de lo necesario. De ahí que el primer paso de todo ensamblador sea corregir las lecturas o descartar los k -mers por debajo del valle del espectro (sección 8.1).

Velvet y la limpieza del grafo. Zerbino y Birney (2008) publicaron Velvet, el primer ensamblador de De Bruijn de uso generalizado para las lecturas muy cortas (25–50 pb) de los primeros secuenciadores Solexa/Illumina. Su aportación principal fue un conjunto de operaciones de limpieza: fusionar los caminos no ramificados, podar las puntas cortas y de baja cobertura, eliminar las burbujas con un recorrido del grafo que compara caminos paralelos y conserva el mejor cubierto, y usar después la información de las lecturas pareadas para atravesar repeticiones. En simulaciones de genomas procariotas, Velvet alcanzó *contigs* con N50 de hasta 50 kb usando lecturas pareadas; con datos reales de Solexa sin pares, unos 8 kb en una bacteria.

⚠ Podar demasiado borra biología

Las operaciones de limpieza se basan en la cobertura: lo poco cubierto se considera error. Pero hay biología genuina con poca cobertura: alelos de una población minoritaria en un metagenoma, plásmidos de baja copia, regiones ricas en GC infrarrepresentadas por la PCR (??). Y la eliminación de burbujas funde los dos haplotipos de un diploide en un mosaico que no existe en ninguna célula. Si su pregunta biológica depende de variantes de baja frecuencia o de la fase de los haplotipos, un ensamblaje “limpio” puede haber borrado justamente la respuesta.

8.2.6 Repeticiones, pares y lecturas largas

El análisis del ejemplo de juguete se generaliza. Una repetición de longitud R solo puede resolverse si alguna pieza de información la *atraviesa* de lado a lado, anclada en secuencia única a ambos lados:

- un k -mer con $k > R + 1$, lo que exige lecturas más largas que R y buena cobertura con k grande (figura 8.3);
- un par de lecturas cuya distancia (el tamaño del inserto) sea mayor que R , de modo que una lectura caiga a cada lado;
- una lectura larga, de PacBio u Oxford Nanopore, que contenga la repetición completa.

Por eso las limitaciones del ensamblaje con lecturas cortas no se superan secuenciando *más*, sino secuenciando *más largo*. La teoría de Lander-Waterman (??) predice un único *contig* para una bacteria a 30×; en la práctica se obtienen decenas, uno por cada repetición más larga que la lectura, como los operones de ARN ribosómico de unos 5 kb, que las bacterias suelen tener en varias copias. Nagarajan y Pop (2013) ofrecen una revisión muy clara de cómo interactúan cobertura, longitud de lectura y repeticiones.

🔗 De los puentes de Königsberg al genoma

La idea de convertir lecturas en palabras y palabras en aristas surgió en el contexto de la secuenciación por hibridación, una tecnología que nunca llegó a imponerse. Idury y Waterman (1995) la trasladaron al ensamblaje de fragmentos, y Pevzner et al. (2001) la llevaron a su forma madura con EULER, que, según sus autores, en lugar de enmascarar las repeticiones las usaba como herramienta de ensamblaje. Durante unos años la idea pareció una curiosidad académica, porque las lecturas de Sanger (??) eran largas y los ensambladores OLC funcionaban bien. Cuando llegaron las lecturas de 36 pb de Solexa, calcular N^2 solapamientos entre cientos de millones de lecturas se volvió imposible, y el grafo de De Bruijn pasó a ser la estructura dominante.

OLC: lecturas como nodos, camino hamiltoniano (NP-completo).
De Bruijn: k -mers como aristas, camino euleriano (lineal). Las repeticiones más largas que k hacen que el camino no sea único; los ensambladores entregan los tramos sin ambigüedad (unitigs).

8.3 Ensamblaje con SPAdes y evaluación con QUAST

El rompecabezas sin la foto de la caja

Cuando termina un rompecabezas de mil piezas, sabe si lo hizo bien porque compara el resultado con la foto de la caja. Ahora imagine que no hay caja. Puede contar cuántos bloques sueltos le quedaron y medir el más grande, pero eso no le dice si algún bloque está mal armado: dos piezas de cielo azul encajan igual de bien en muchos sitios, y forzarlas produce un bloque grande y falso. Para juzgar su trabajo necesita otras pistas: que no le sobren ni le falten piezas de los bordes, que ciertas piezas que sabe que existen (la firma del pintor, el sol) aparezcan una sola vez, que los colores de cada unión sean coherentes. Evaluar un ensamblaje es exactamente esto: combinar medidas de tamaño, de exactitud y de completitud, sabiendo que ninguna basta por sí sola.

8.3.1 Un solo k no basta

La sección 8.2 dejó un dilema abierto. Un k grande resuelve más repeticiones, pero la ecuación (8.2) nos dice que reduce la cobertura efectiva, y con ella aumenta la probabilidad de que algún k -mer del genoma no aparezca en ninguna lectura. Cada k -mer ausente corta el camino del grafo y divide un *contig* en dos. Por el mismo argumento de Poisson del ??,

$$\mathbb{P}(\text{un } k\text{-mer del genoma no se observa}) = e^{-\lambda_k}, \quad \lambda_k = c \frac{L - k + 1}{L} (1 - e)^k. \quad (8.11)$$

DONDE

$e^{-\lambda_k}$	Fracción esperada de k -mers del genoma que faltan en el grafo.
λ_k	Cobertura efectiva de k -mers (ecuación (8.2)).

Con cobertura alta el efecto es invisible, pero la cobertura no es uniforme. Una región que por sesgo de GC (??) recibe solo 10× con lecturas de 150 pb y 0,5 % de error pierde el 0,04 % de sus k -mers con $k = 21$, pero el 3,5 % con $k = 77$ y el 43 % con $k = 127$ (figura 8.7). En un genoma de 5 Mb con esa cobertura, son unos 2000, 175 000 y más de dos millones de k -mers ausentes, respectivamente.

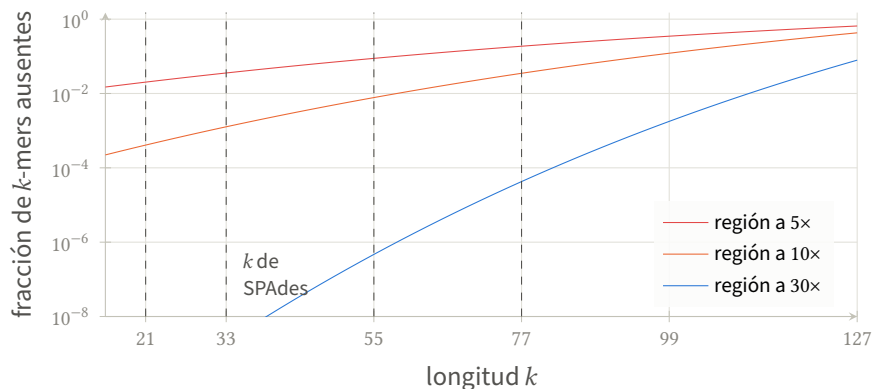


Figura 8.7. Fracción de k -mers del genoma que no aparecen en ninguna lectura (ecuación (8.11)) para lecturas de 150 pb con 0,5 % de error. En una región bien cubierta (30×), incluso $k = 77$ deja menos de un k -mer ausente cada 20 000; en una región a 5×, un k grande fragmenta el grafo sin remedio. Las líneas discontinuas marcan los valores de k que SPAdes usa por defecto con lecturas de 150 pb: los pequeños mantienen conectadas las regiones pobres y los grandes resuelven repeticiones en las ricas.

8.3.2 SPAdes

Bankevich et al. (2012) diseñaron SPAdes para un caso extremo: el ensamblaje de genomas bacterianos amplificados a partir de *una sola célula*, donde la cobertura varía en órdenes de magnitud a lo largo del genoma y abundan los errores y las lecturas quiméricas. En ese escenario ningún k fijo funciona, y la respuesta de SPAdes fue construir el grafo con *varios* valores de k de forma iterativa (figura 8.8). Primero ensambla con un k pequeño, que conecta las regiones de baja cobertura; luego usa los *contigs* obtenidos como lecturas adicionales, largas y sin huecos, al construir el grafo con un k mayor, que separa repeticiones. El resultado combina lo mejor de ambos extremos. Después, SPAdes simplifica el grafo (puntas, burbujas, conexiones quiméricas) y usa las lecturas pareadas para estimar distancias entre aristas y atravesar repeticiones. Aunque nació para células individuales, sus autores mostraron que también mejoraba a Velvet y SOAPdenovo en datos convencionales, y hoy es uno de los ensambladores más usados para genomas bacterianos con lecturas cortas.

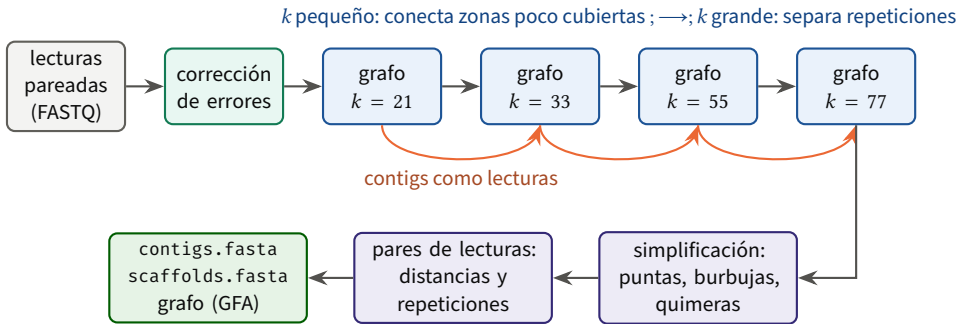


Figura 8.8. Esquema del ensamblaje multi-*k* de SPAdes. Tras corregir las lecturas, se construye el grafo de De Bruijn con un *k* pequeño; sus *contigs* se reinyectan como lecturas largas y sin errores en el grafo del siguiente *k* (flechas naranjas), de modo que las regiones pobremente cubiertas que el *k* grande perdería quedan representadas. El grafo final se simplifica y se recorre con la información de los pares. Además de las secuencias, SPAdes escribe el grafo de ensamblaje en formato GFA, que puede inspeccionarse con visores como Bandage.

Ensamblar un aislado bacteriano con SPAdes

```
# lecturas pareadas ya recortadas (capítulo 6)
spades.py --isolate -1 cepa_R1.fq.gz -2 cepa_R2.fq.gz \
  -k 21,33,55,77 -t 8 -m 32 -o spades_cepa

ls spades_cepa/
# contigs.fasta scaffolds.fasta assembly_graph.gfa spades.log ...

# contigs muy cortos o poco cubiertos suelen ser basura: filtrar
seqkit seq -m 500 spades_cepa/contigs.fasta > contigs_500.fasta
```

Los nombres de los *contigs* de SPAdes llevan la información más útil para una primera inspección, por ejemplo `>NODE_1_length_812345_cov_41.2`: longitud y cobertura de *k*-mers. Un *contig* con cobertura cercana al doble de la mediana es probablemente una repetición colapsada de dos copias; uno con cobertura diez veces mayor puede ser un plásmido multicopia; y uno con cobertura muy baja, un contaminante o un artefacto.

8.3.3 Lecturas largas: el regreso del solapamiento

Las lecturas largas cambiaron de nuevo el equilibrio. Con lecturas de decenas de kilobases hay pocas lecturas y muy largas, y calcular solapamientos vuelve a ser

factible; además, una lectura larga atraviesa repeticiones que ningún k de lecturas cortas resolvería. La contrapartida es que las primeras lecturas largas tenían tasas de error del 10 al 15 %, incompatibles con k -mers exactos.

- **Canu** (Koren et al., 2017), sucesor del Celera Assembler, es un ensamblador OLC diseñado para lecturas ruidosas de PacBio y Nanopore. Detecta solapamientos con una estrategia adaptativa de *MinHash* ponderada con tf-idf (que da menos peso a los k -mers repetitivos), corrige las lecturas y construye un grafo disperso que evita colapsar repeticiones y haplotipos divergentes. Obtuvo *contigs* con NG50 superior a 21 Mb en datos de PacBio humanos y de *Drosophila melanogaster*.
- **Flye** (Kolmogorov et al., 2019) vuelve a la idea de los grafos de repeticiones: genera caminos arbitrarios (*disjointigs*) a través de un grafo de repeticiones desconocido y reconstruye a partir de ellos un grafo exacto. Frente a cinco ensambladores de referencia fue un orden de magnitud más rápido y casi duplicó la contigüidad (NGA50) del ensamblaje humano.
- **hifiasm** (Cheng et al., 2021) aprovecha las lecturas HiFi, largas y con exactitud superior al 99 %, para construir un *grafo de ensamblaje con fase*, que conserva la contigüidad de *todos* los haplotipos en lugar de colapsarlos en un consenso. Ensambló, entre otros, el genoma hexaploide de unos 30 Gb de la secuoya de California.

La culminación de esta línea fue el primer genoma humano completo de telómero a telómero. Nurk et al. (2022) combinaron lecturas HiFi y ultralargas de Nanopore de la línea celular CHM13, prácticamente homocigota, para producir una secuencia de 3055 millones de pares de bases sin huecos en todos los cromosomas salvo el Y. El ensamblaje T2T-CHM13 corrigió errores de las referencias previas y añadió casi 200 millones de pares de bases (el 8 % del genoma que faltaba), con 1956 predicciones génicas, 99 de ellas codificantes de proteínas: los centrómeros, las duplicaciones segmentarias recientes y los brazos cortos de los cinco cromosomas acrocéntricos, las regiones que ninguna lectura corta había podido atravesar.

8.3.4 Métricas de contigüidad: N50 y sus parientes

Un ensamblaje es una lista de secuencias de longitudes $\ell_1 \geq \ell_2 \geq \dots \geq \ell_n$ (ordenadas de mayor a menor), con longitud total $A = \sum_i \ell_i$. La media de las longitudes es una mala medida de calidad: la dominan los miles de *contigs* diminutos

que casi todo ensamblaje produce. La comunidad adoptó en su lugar una medida ponderada por longitud.

Definición 8.4. (Nx , Lx y NGx)

Para $0 < x \leq 100$, el Nx es la longitud ℓ_j del *contig* en el que la suma acumulada alcanza el $x\%$ del total:

$$Nx = \ell_{j^*}, \quad j^* = \min \left\{ j : \sum_{i=1}^j \ell_i \geq \frac{x}{100} A \right\}, \quad Lx = j^*. \quad (8.12)$$

El NGx (y el LGx) se define igual, sustituyendo A por el tamaño del genoma G , conocido o estimado (ecuación (8.6)). Si el ensamblaje no alcanza el $x\%$ de G , el NGx no está definido.

DONDE

ℓ_i	Longitud del i -ésimo <i>contig</i> más largo.
A	Longitud total del ensamblaje.
j^*	Número mínimo de <i>contigs</i> que, tomados de mayor a menor, suman el $x\%$.
Lx	Número de <i>contigs</i> (el L_{50} es cuántos hacen falta para cubrir la mitad).

Una lectura equivalente e intuitiva: el N_{50} es la longitud tal que la mitad de las bases del ensamblaje están en *contigs* de esa longitud o mayores. Si se escoge una base del ensamblaje al azar, hay una probabilidad del 50 % de que caiga en un *contig* de al menos N_{50} bases. Esa interpretación sugiere una medida sin umbral arbitrario: la longitud esperada del *contig* que contiene una base elegida al azar,

$$\text{auN} = \sum_i \frac{\ell_i}{A} \ell_i = \frac{\sum_i \ell_i^2}{\sum_i \ell_i}, \quad (8.13)$$

que es además el área bajo la curva Nx (dividida por 100) cuando x recorre de 0 a 100 (figura 8.9).

DONDE

ℓ_i/A	Probabilidad de que una base elegida al azar pertenezca al <i>contig</i> i .
auN	Longitud media del <i>contig</i> “visto desde una base”; resume toda la curva Nx .

Ejemplo 8.2. (Calcular N50, NG50 y auN a mano)

Un ensamblaje tiene diez *contigs* de 320, 250, 180, 120, 90, 60, 40, 25, 10 y 5 kb; total $A = 1100$ kb. El genoma estimado con el espectro de k -mers mide $G = 1300$ kb.

- N_{50} : la mitad de A es 550 kb. Acumulando, 320 y luego $320 + 250 = 570 \geq 550$. Por tanto $N_{50} = 250$ kb y $L_{50} = 2$.
- NG_{50} : la mitad de G es 650 kb. Hace falta llegar a $320 + 250 + 180 = 750$: $NG_{50} = 180$ kb y $LG_{50} = 3$. El NG_{50} es menor porque penaliza los 200 kb del genoma que el ensamblaje no contiene.
- N_{90} : el 90 % de A son 990 kb, que se alcanzan en el sexto *contig*: $N_{90} = 60$ kb.
- auN : $\sum \ell_i^2 = 320^2 + 250^2 + \dots + 5^2 = 225\,750$, y $225\,750/1100 = 205,2$ kb.

Ahora supongamos que el ensamblador une por error los dos *contigs* mayores a través de una repetición. El nuevo *contig* de 570 kb supera por sí solo la mitad del total: el N_{50} salta a 570 kb y el L_{50} baja a 1. *Un error de ensamblaje ha más que duplicado el N_{50} .*

n50.py

```
def nx(longitudes, x=50, G=None):
    """Devuelve (Nx, Lx); con G, (NGx, LGx)."""
    ls = sorted(longitudes, reverse=True)
    meta = x / 100 * (G if G else sum(ls))
    acum = 0
    for i, l in enumerate(ls, start=1):
        acum += l
        if acum >= meta:
            return l, i
    return 0, None          # el ensamblaje no alcanza x% de G

def auN(longitudes):
    return sum(l * l for l in longitudes) / sum(longitudes)

contigs = [320, 250, 180, 120, 90, 60, 40, 25, 10, 5]    # kb
print(nx(contigs), nx(contigs, G=1300), round(auN(contigs), 1))
# (250, 2) (180, 3) 205.2
```

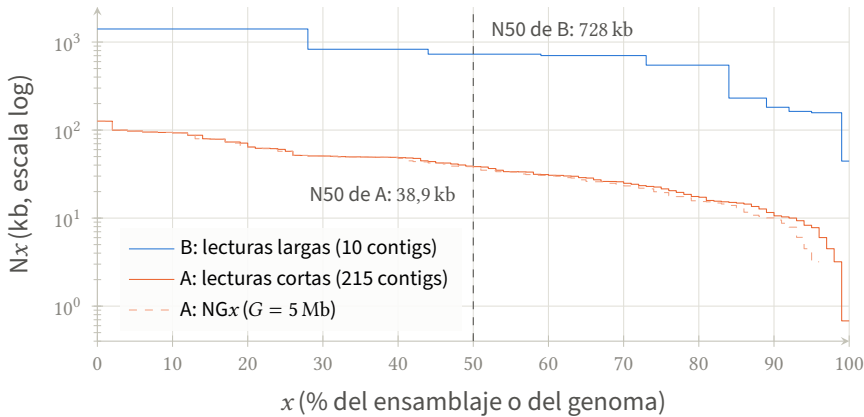


Figura 8.9. Curvas Nx de dos ensamblajes simulados del mismo genoma bacteriano de 5 Mb. Cada curva escalonada indica, para cada x , la longitud del *contig* en el que la suma acumulada alcanza el $x\%$; la ordenada en $x = 50$ es el N50. El ensamblaje de lecturas largas (azul) tiene un N50 casi veinte veces mayor que el de lecturas cortas (naranja). La curva NGx discontinua cae a cero antes de $x = 100$ porque el ensamblaje A solo contiene el 97 % del genoma. Comparar curvas completas es más informativo que comparar un único punto. Generado con `figuras/cap08/generar.py`.

⚠ El N50 mide tamaño, no verdad

El ejemplo anterior muestra el defecto fundamental de todas las métricas de contigüidad: premian las uniones, correctas o no. Un ensamblador agresivo que atraviesa repeticiones sin evidencia obtiene mejores N50 que uno prudente. Además, el N50 depende del umbral de longitud mínima: eliminar los *contigs* de menos de 500 pb reduce A y puede *aumentar* el N50 sin cambiar nada del ensamblaje. Nunca compare N50 sin comparar también la exactitud, y prefiera el NG50, que usa un denominador fijo.

8.3.5 Andamiaje

Los *contigs* terminan donde el grafo es ambiguo, pero a menudo sabemos qué *contig* sigue a cuál aunque no conozcamos la secuencia intermedia. El *andamiaje* (*scaffolding*) ordena y orienta *contigs* usando información de largo alcance: pares de lecturas con insertos largos, lecturas largas, mapas ópticos o contactos de cromatina (Hi-C). El resultado, un *scaffold*, es una secuencia con tramos de N que representan huecos de tamaño estimado. Si una lectura de un par cae a una distancia a del final

del *contig* C_1 y su compañera a una distancia b del inicio de C_2 , y el tamaño medio del inserto es μ , el hueco estimado es

$$\hat{g} = \mu - a - b, \quad (8.14)$$

y promediando muchos pares se reduce la incertidumbre. Un $\hat{g}$ negativo indica que los *contigs* se solapan (a menudo por una repetición colapsada en sus extremos).

DONDE

μ	Tamaño medio del inserto de la biblioteca (distancia entre los extremos externos del par).
a, b	Distancias de cada lectura del par al extremo correspondiente de su <i>contig</i> .
$\hat{g}$	Longitud estimada del hueco que se rellenará con N.

8.3.6 Exactitud: QUAST

Cuando existe un genoma de referencia cercano, podemos medir la exactitud directamente alineando los *contigs* contra él. QUAST (Gurevich et al., 2013) se ha convertido en el estándar para ello. Con referencia, además de las métricas de contigüidad, calcula la fracción del genoma cubierta, la razón de duplicación, los desajustes e *indels* por cada 100 kb y, sobre todo, los *errores de ensamblaje* (*misassemblies*): puntos de un *contig* en los que el tramo izquierdo y el derecho se alinean en la referencia a más de 1 kb de distancia, en hebras opuestas o en cromosomas distintos (figura 8.10). Con esos puntos corta los *contigs* y recalcula la contigüidad: el NGA_{50} es el NG_{50} de los bloques alineados correctamente, una medida que ya no premia las uniones falsas. Sin referencia, QUAST sigue siendo útil para comparar varios ensamblajes de los mismos datos con las métricas de la sección anterior.

Evaluación con QUAST, BUSCO y Merqury

```
# contigüidad y exactitud frente a una referencia cercana
quast.py contigs_500.fasta otro_ensamblaje.fasta \
  -r referencia.fasta -g genes.gff -t 8 -o quast_cepa

# completitud génica con ortólogos de copia única
busco -i contigs_500.fasta -m genome -l bacteria_odb10 \
  -c 8 -o busco_cepa

# exactitud sin referencia a partir de los k-mers de las lecturas
```

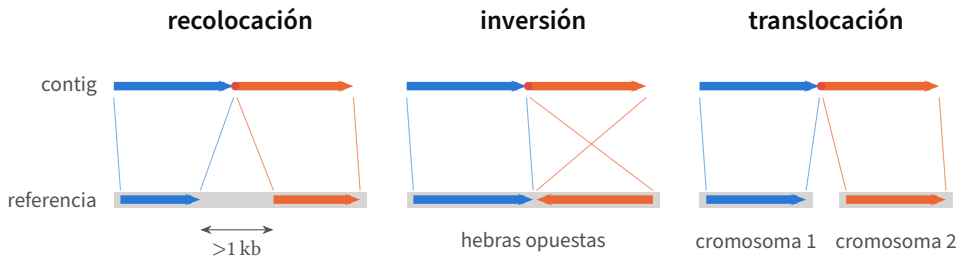


Figura 8.10. Los tres tipos de errores de ensamblaje que QUAST detecta al alinear un *contig* (arriba) contra la referencia (abajo, gris). El punto rojo es el punto de ruptura. **Recolocación:** los dos tramos se alinean en la misma hebra pero separados (o solapados) por más de 1 kb. **Inversión:** el tramo derecho se alinea en la hebra opuesta. **Translocación:** los tramos proceden de cromosomas (o replicones) distintos. Casi siempre el origen es una repetición atravesada por el camino equivocado del grafo (figura 8.6), aunque algunos “errores” son variación estructural real entre la muestra y la referencia.

```
meryl k=21 count output lect.meryl cepa_R1.fq.gz cepa_R2.fq.gz
mercury.sh lect.meryl contigs_500.fasta mercury_cepa
```

8.3.7 Completitud y exactitud sin referencia

Para la mayoría de los organismos no existe una referencia. Dos familias de métodos cubren ese hueco.

Genes que no pueden faltar. BUSCO (Simão et al., 2015) se basa en una observación evolutiva: en cada linaje hay genes que están presentes en casi todas las especies y casi siempre en una sola copia (*benchmarking universal single-copy orthologs*). Si buscamos esos genes en un ensamblaje, cada uno debería aparecer exactamente una vez. BUSCO clasifica cada ortólogo como completo y de copia única (S), completo y duplicado (D), fragmentado (F) o ausente (M). Un buen ensamblaje bacteriano suele superar el 95 % de completos; muchos fragmentados sugieren errores de consenso o *contigs* cortos; muchos duplicados, que los dos haplotipos de un diploide quedaron separados sin querer o que hay contaminación con un organismo emparentado. La versión 5 amplió los conjuntos de datos (sincronizados con OrthoDB v10), incorporó la selección automática del linaje por ubicación filogenética, lo que permite evaluar genomas de metagenomas de origen desconocido, y admite eucariotas, procariotas y virus (Manni et al., 2021).

k -mers como testigos. Merqury (Rhie et al., 2020) compara los k -mers del ensamblaje con los de las lecturas. Supongamos que el ensamblaje tiene K_{asm} k -mers y que K_{solo} de ellos no aparecen en ninguna lectura (errores de consenso). Si los errores son independientes y afectan a una base con probabilidad ε , un k -mer es correcto con probabilidad $(1 - \varepsilon)^k$, así que

$$(1 - \varepsilon)^k \approx 1 - \frac{K_{\text{solo}}}{K_{\text{asm}}} \implies \hat{\varepsilon} = 1 - \left(1 - \frac{K_{\text{solo}}}{K_{\text{asm}}}\right)^{1/k}, \quad \text{QV} = -10 \log_{10} \hat{\varepsilon}. \quad (8.15)$$

DONDE

K_{asm}	Número de k -mers del ensamblaje.
K_{solo}	k -mers del ensamblaje ausentes de las lecturas.
$\hat{\varepsilon}$	Tasa de error por base estimada del ensamblaje.
QV	Calidad de consenso en escala Phred (la misma escala del ??).

Con $k = 21$, un ensamblaje bacteriano de 5 millones de k -mers con 250 k -mers exclusivos tiene $\hat{\varepsilon} = 2,4 \times 10^{-6}$ y $\text{QV} = 56$: un error cada 420 000 bases. Con 5000 k -mers exclusivos, $\text{QV} = 43$. Merqury también mide la completitud como la fracción de k -mers “fiables” de las lecturas (los que superan el valle del espectro) que aparecen en el ensamblaje, y dibuja espectros coloreados según cuántas veces aparece cada k -mer en el ensamblaje, lo que revela de un vistazo si se perdió un haplotipo o se duplicó.

Un ensamblaje se juzga en tres ejes: contigüidad (N50, NG50, auN), exactitud (errores de QUAST, NGA50, QV de Merqury) y completitud (fracción del genoma, BUSCO, k -mers). Mejorar uno a costa de otro es fácil; mejorar los tres a la vez es lo que distingue un buen ensamblaje.

8.4 Anotación genómica



Encontrar palabras en un texto sin espacios

Lea esta línea:

ELQUEMUCHOABARCAPOCOAPRIETAYXQWZKTRBLAMPENCIAHJXV

Sin que nadie le haya dicho dónde termina cada palabra, usted separa el refrán al principio y reconoce que el final es ruido. Lo consigue porque conoce el español: sabe qué combinaciones de letras son frecuentes (QUE, CHO), cuáles son casi imposibles (XQWZ) y qué forma tiene una palabra. Un programa de predicción génica hace lo mismo con el ADN. No sabe biología; sabe estadística: aprende cómo “suena” el ADN codificante de un organismo y busca en el genoma los tramos que suenan así, respetando una gramática (un gen empieza con un codón de inicio, no puede contener codones de parada en su marco y, en eucariotas, sus intrones empiezan y terminan con señales características).

8.4.1 Qué significa anotar

Anotar un genoma tiene dos niveles. La *anotación estructural* asigna coordenadas: dónde están los genes codificantes, sus exones y sus codones de inicio y parada; los genes de ARN ribosómico y de transferencia; las repeticiones y los elementos móviles. La *anotación funcional* asigna significado: nombre del producto génico, función enzimática, dominios, términos de ontología. El resultado se guarda en formatos como GFF3 o GenBank (??). La figura 8.11 resume el flujo de trabajo, que difiere sobre todo en la complejidad de la predicción de genes: en procariotas los genes son casi siempre marcos de lectura continuos y densos (alrededor del 88 % del genoma de *Escherichia coli* codifica proteínas); en eucariotas están fragmentados en exones separados por intrones y dispersos en un mar de ADN no codificante y repetitivo.

8.4.2 Genes procariotas: marcos abiertos de lectura

Definición 8.5. (Marco abierto de lectura (ORF))

Un *marco abierto de lectura* (*open reading frame*, ORF) es un tramo de ADN que, leído en uno de los seis marcos posibles (tres por hebra), comienza en un codón de inicio (típicamente ATG, y en bacterias también GTG o TTG) y continúa sin codones de parada (TAA, TAG, TGA) hasta el primero que aparece en ese marco.

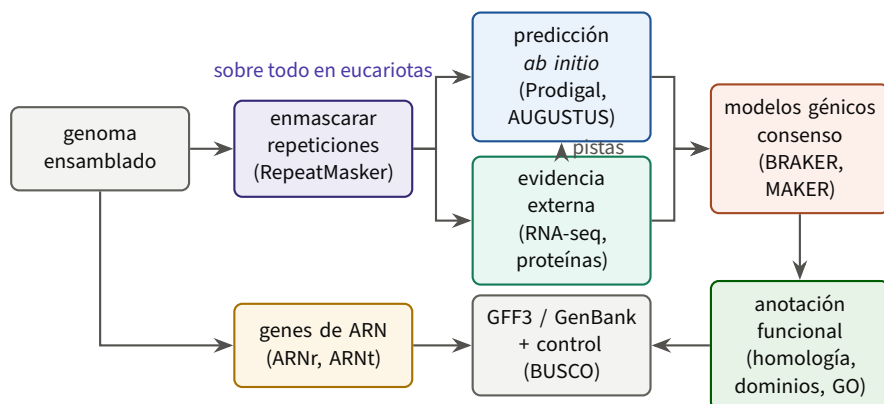


Figura 8.11. Flujo típico de anotación. Las repeticiones se enmascaran primero para que sus marcos de lectura (transposasas, transcriptasas inversas) no se confundan con genes del huésped. La predicción *ab initio* usa solo la secuencia y un modelo estadístico entrenado para la especie; la evidencia externa (transcritos de RNA-seq alineados, proteínas de especies cercanas) se incorpora como “pistas” que el predictor debe respetar o como modelos independientes que después se combinan. Los genes de ARN estructurales se buscan con modelos propios. Por último, a cada gen se le asigna una función por homología y el conjunto se valida, por ejemplo, con BUSCO en modo proteína. En procariotas, herramientas como Prokka y Bakta automatizan todo el flujo.

Todo gen codificante bacteriano es un ORF, pero no todo ORF es un gen: en cualquier secuencia aparecen ORFs por azar. ¿Cuán largos? Si los codones fueran independientes, cada uno sería de parada con probabilidad p , y la longitud de un ORF aleatorio (en codones) sería geométrica:

$$P(\text{ORF aleatorio} \geq n \text{ codones}) = (1 - p)^n, \quad p = \sum_{xyz \in \{\text{TAA, TAG, TGA}\}} f_x f_y f_z. \quad (8.16)$$

DONDE

p	Probabilidad de que un codón al azar sea de parada, dada la composición de bases.
f_x	Frecuencia de la base x en el genoma.
n	Longitud del ORF en codones.

En el genoma de *E. coli* K-12 (50,8 % de GC), $p = 0,0456$, aproximadamente un codón de parada cada 22. La probabilidad de que un ORF aleatorio supere los 100 codones es $(1 - p)^{100} = 0,0094$, y la de que supere los 300, 8×10^{-7} . La figura 8.12

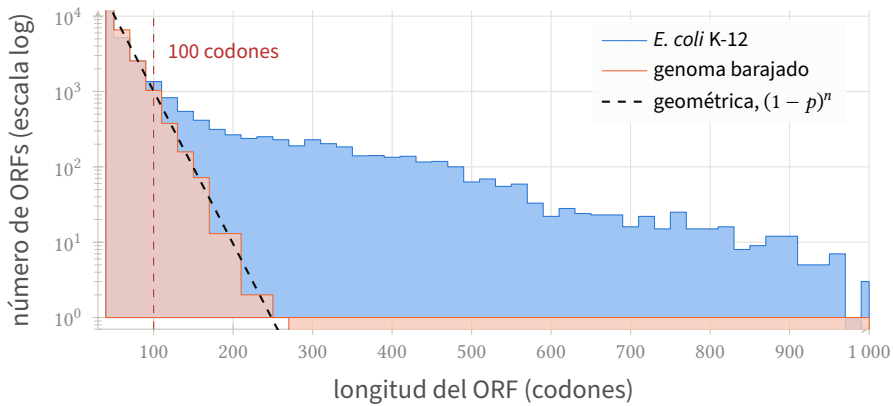


Figura 8.12. Distribución de longitudes de los ORFs (de ATG a codón de parada, seis marcos, ≥ 30 codones) en el genoma de *E. coli* K-12 MG1655 (azul) y en el mismo genoma barajado base a base (naranja). El genoma barajado sigue la ley geométrica de la ecuación (8.16) (línea discontinua): su ORF más largo tiene 251 codones. El genoma real muestra una cola de miles de ORFs largos, los genes. Por debajo de unos 100 codones las dos distribuciones se confunden: allí la longitud ya no basta y hace falta el contenido de la secuencia. Datos: NC_000913.3; generado con `figuras/cap08/generar.py`.

compara los ORFs reales del genoma de *E. coli* con los de una versión barajada con la misma composición: el genoma barajado no produce *ningún* ORF de 300 codones o más, mientras que el real tiene 2030. Los ORFs largos son genes casi con certeza.

8.4.3 El sabor del ADN codificante: modelos de Márkov

Para los ORFs cortos, la longitud no basta: hay que mirar su composición. El ADN codificante tiene una periodicidad de tres bases que el no codificante no tiene, porque cada posición del codón está sometida a restricciones distintas (la tercera es la más libre y refleja el sesgo de uso de codones de cada especie). La forma más simple de explotarlo es un modelo de codones independientes.

Definición 8.6. (Puntuación log-odds de codones)

Sea $P_{\text{cod}}(c)$ la frecuencia del codón c en genes conocidos y $P_{\text{nc}}(c) = f_{c_1}f_{c_2}f_{c_3}$ su probabilidad bajo un modelo nulo de bases independientes. La puntuación de un ORF $x = c_1c_2 \cdots c_n$ es

$$S(x) = \frac{1}{n} \sum_{i=1}^n \log_2 \frac{P_{\text{cod}}(c_i)}{P_{\text{nc}}(c_i)} \quad (\text{bits por codón}). \quad (8.17)$$

DONDE

$P_{\text{cod}}(c)$	Probabilidad del codón c en regiones codificantes de la especie.
$P_{\text{nc}}(c)$	Probabilidad del mismo trinucleótido bajo el modelo nulo.
$S(x)$	Evidencia media, en bits por codón, de que x es codificante.

Es la misma lógica de razón de verosimilitudes de las matrices de sustitución (??). ¿De dónde salen las frecuencias P_{cod} de un genoma recién ensamblado, sin genes conocidos? De la figura 8.12: los ORFs de más de 300 codones son genes casi con certeza, así que podemos *autoentrenar* el modelo con ellos. En *E. coli*, los 2030 ORFs largos aportan casi un millón de codones. Los codones más favorecidos resultan ser CTG (+1,76 bits, el codón de leucina preferido por la bacteria), GAA (+1,32) y AAA (+1,09); los más desfavorecidos, los codones de arginina raros AGG (−3,79) y AGA (−2,83).

Ejemplo 8.3. (Puntuar un fragmento)

Con el modelo autoentrenado en *E. coli*, el fragmento ATG AAA CGC ATT AGC ACC ACC ATT recibe

$$S = \frac{1}{8} (0,83+1,09+0,48+0,98+0,03+0,65+0,65+0,98) = \frac{5,69}{8} = 0,71 \text{ bits/codón},$$

un valor típico de genes muy expresados. Un fragmento del mismo tamaño que contuviera AGG, AGA y CGA acumularía casi −9 bits solo por esos tres codones. Aplicado a los ORFs de 60 a 149 codones, la región de la figura 8.12 donde las dos distribuciones se confunden, el 27,8 % de los ORFs reales supera 0,1 bits/codón, frente al 0,22 % de los ORFs del genoma barajado (figura 8.13).

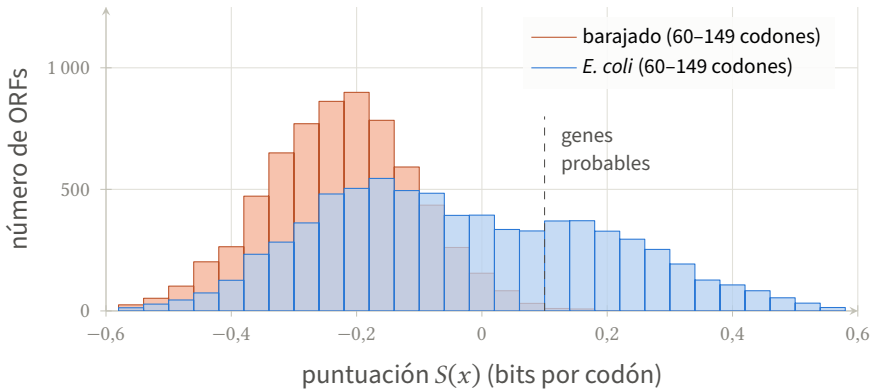


Figura 8.13. Distribución de la puntuación de la ecuación (8.17) para los ORFs de 60 a 149 codones de *E. coli* (azul) y del genoma barajado (naranja), con un modelo autoentrenado con los ORFs de más de 300 codones. Los ORFs barajados forman una única campana negativa; los reales son una mezcla de una campana parecida (ORFs espurios, a menudo en otro marco de un gen verdadero) y una cola positiva de genes cortos. Incluso este modelo de orden cero separa buena parte de los genes; los predictores reales usan cadenas de Márkov de orden superior dependientes de la fase del codón.

El modelo de codones independientes ignora el contexto. Los predictores reales usan *cadenas de Márkov de orden m dependientes de la fase*: la probabilidad de cada base depende de las m anteriores y de la posición que ocupa dentro del codón,

$$\log P_{\text{cod}}(x) = \sum_i \log P(x_i \mid x_{i-m} \cdots x_{i-1}, \phi(i)), \quad \phi(i) \in \{1, 2, 3\}. \quad (8.18)$$

DONDE

m	Orden de la cadena: cuántas bases previas se usan como contexto (típicamente 5).
$\phi(i)$	Posición de la base i dentro del codón (fase).

Con $m = 5$ hay $3 \times 4^5 \times 3 = 9216$ parámetros independientes, y los contextos raros se estiman mal. Glimmer resolvió ese problema con *modelos de Márkov interpolados*, que combinan órdenes de 0 a 8 con pesos que dependen de cuántas veces se observó cada contexto; su versión 3 redujo drásticamente los falsos positivos manteniendo una sensibilidad del 99 % y mejoró la identificación de los codones de inicio (Delcher et al., 2007). GeneMark.hmm (Lukashin & Borodovsky, 1998) insertó los modelos de Márkov de GeneMark en un modelo oculto de Márkov en el que los límites

de los genes son transiciones entre estados ocultos, y usó un patrón del sitio de unión al ribosoma para precisar los codones de inicio; sus autores observaron alta exactitud incluso con cadenas de orden cero, uno y dos. Prodigal (Hyatt et al., 2010), desarrollado a partir de la experiencia de curación manual de genomas en el Joint Genome Institute, usa programación dinámica sobre todos los pares inicio–parada posibles, con tres objetivos explícitos: mejor estructura génica, mejor sitio de inicio de la traducción y menos falsos positivos. Todos son autoentrenables: aprenden los parámetros del propio genoma que anotan.

8.4.4 Genes eucariotas: modelos ocultos de Márkov generalizados

En eucariotas, un gen es una frase con gramática: región intergénica, codón de inicio, exón, sitio donador (GT), intrón, sitio aceptor (AG), exón... y codón de parada. Un modelo oculto de Márkov (HMM) es la herramienta natural para describir esa gramática: cada *estado* oculto corresponde a un tipo de región, cada estado *emite* bases con su propio modelo (una cadena de Márkov de fase para los exones, otra para los intrones), y las *transiciones* entre estados codifican el orden permitido (figura 8.14). Como un intrón puede interrumpir un codón en cualquiera de sus tres posiciones, hay que distinguir tres estados de intrón según la fase, para que el exón siguiente continúe el marco correctamente.

La anotación de una secuencia $x_1 \dots x_T$ consiste en encontrar la sucesión de estados π^* más probable dada la secuencia, lo que se resuelve con el algoritmo de Viterbi. Llamando $V_j(t)$ a la probabilidad (en logaritmos) del mejor camino que termina en el estado j tras emitir $x_1 \dots x_t$,

$$V_j(t) = \log e_j(x_t) + \max_i [V_i(t-1) + \log a_{ij}], \quad \pi^* = \arg \max_{\pi} \mathbb{P}(x, \pi). \quad (8.19)$$

DONDE

a_{ij}	Probabilidad de transición del estado i al j (p. ej., de exón a sitio donador).
$e_j(x_t)$	Probabilidad de que el estado j emita la base x_t (dado su contexto).
$V_j(t)$	Log-probabilidad del mejor camino de estados que explica $x_1 \dots x_t$ y acaba en j .
π^*	Anotación más probable: la etiqueta (intergénica, exón, intrón...) de cada base.

Es la misma programación dinámica de Needleman-Wunsch (??), ahora sobre estados en lugar de columnas de alineamiento, con coste $O(T|Q|^2)$ para $|Q|$ estados.

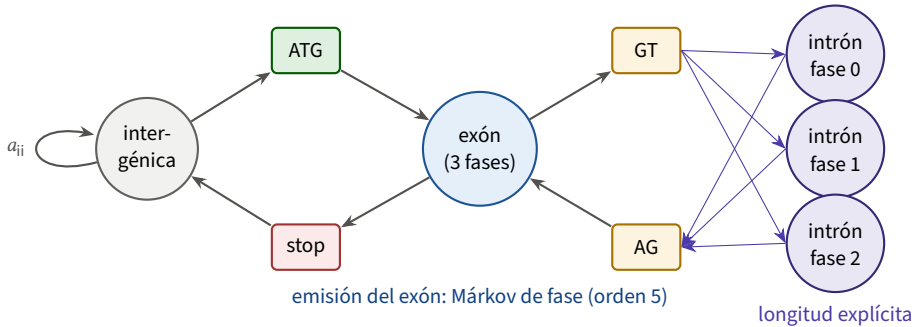


Figura 8.14. Esquema simplificado (una sola hebra) de un HMM generalizado para la estructura de un gen eucariota, en el espíritu de AUGUSTUS. Los estados circulares emiten tramos de secuencia con modelos de contenido propios (el exón, con cadenas de Márkov de fase); los rectangulares, señales cortas (inicio, parada, sitios de empalme donador GT y aceptor AG). Los tres estados de intrón recuerdan en qué posición del codón se interrumpió el exón. En un HMM “generalizado” cada estado de contenido emite un tramo entero cuya longitud sigue una distribución arbitraria, en lugar de la geométrica implícita en un bucle sobre sí mismo. Un modelo completo duplica todo en la hebra opuesta.

Un HMM simple impone, sin embargo, longitudes geométricas a cada región (la probabilidad de seguir en un estado es constante), y las longitudes reales de exones e intrones no son geométricas. Los HMM *generalizados* permiten que cada estado emita un segmento entero con una distribución de longitud arbitraria, a cambio de un coste computacional mayor. AUGUSTUS (Stanke & Waack, 2003) introdujo una nueva forma de modelar la longitud de los intrones (explícita para los cortos, con una cola geométrica para los largos), un nuevo modelo del sitio donador y una estimación de parámetros dependiente del contenido de GC. Sus autores partían de un diagnóstico preciso: los predictores de entonces funcionaban bien en secuencias cortas con un solo gen, pero en secuencias largas con un número desconocido de genes predecían muchos exones falsos. En ese escenario, AUGUSTUS predijo correctamente muchos más genes humanos y de *Drosophila* que los programas con los que se comparó, y a la vez fue más específico. Hoy es uno de los predictores eucariotas más usados.

8.4.5 Enmascarar repeticiones

Antes de predecir genes en un eucariota hay que ocuparse de las repeticiones. Casi la mitad del genoma humano, y porcentajes aún mayores en muchas plantas,

está formada por elementos transponibles y sus restos. Muchos de ellos contienen ORFs genuinos (transposasas, transcriptasas inversas) que un predictor *ab initio* anotaría como genes del huésped, inflando el número de genes con miles de falsos positivos. RepeatMasker (Tarailo-Graovac & Chen, 2009) busca en el genoma las secuencias de una biblioteca de repeticiones conocidas (Dfam, Repbase) o construida *de novo* para la especie, y las marca. El marcado puede ser *duro* (sustituir por N) o *blando* (*soft-masking*: pasar a minúsculas); el blando es preferible, porque permite que el predictor evite iniciar genes en esas regiones sin impedir que un exón verdadero se extienda sobre un fragmento repetitivo.

8.4.6 Evidencia: RNA-seq y proteínas

Los predictores *ab initio* aciertan bien la estructura de genes típicos, pero fallan en los extremos (exones terminales, regiones no traducidas), en los genes atípicos y en el empalme alternativo. La evidencia experimental corrige esas debilidades. Las lecturas de RNA-seq alineadas con un mapeador que admite empalmes indican directamente dónde están los intrones: cada lectura dividida entre dos exones es una “pista” de intrón. Las proteínas de especies emparentadas, alineadas contra el genoma, marcan exones codificantes conservados. Yandell y Ence (2012) revisan cómo los flujos de anotación combinan estas fuentes y cómo medir la concordancia entre un modelo génico y la evidencia que lo apoya.

BRAKER automatiza la combinación: usa la evidencia para entrenar GeneMark sin supervisión, genera con él un conjunto de genes de entrenamiento para AUGUSTUS y predice los genes finales con AUGUSTUS respetando las pistas. La versión BRAKER2 (Brůna et al., 2021) permite hacerlo con una base de datos de proteínas de cualquier distancia evolutiva, incluso sin RNA-seq, mediante GeneMark-EP+.

Anotación eucariota con enmascarado y BRAKER

```
# 1) biblioteca de repeticiones de la especie y enmascarado blando
BuildDatabase -name especie genoma.fasta
RepeatModeler -database especie -threads 16
RepeatMasker -lib especie-families.fa -xsmall -pa 16 genoma.fasta

# 2) predicción con evidencia de proteínas y RNA-seq alineado
braker.pl --genome=genoma.fasta.masked --softmasking \
  --prot_seq=orthodb_proteinas.fa --bam=rnaseq.bam \
  --threads 16 --workingdir=braker_out

# 3) completitud del conjunto de proteínas predicho
```

```
busco -i braker_out/braker.aa -m proteins \
-l eukaryota_odb10 -o busco_prot
```

8.4.7 Anotación procariota automatizada: Prokka y Bakta

En bacterias y arqueas el flujo completo puede automatizarse. Prokka (Seemann, 2014) encadena herramientas especializadas: Prodigal para las regiones codificantes y otros programas para los genes de ARNr, ARNt, ARN no codificantes y péptidos señal; asigna funciones buscando las proteínas predichas en bases de datos jerárquicas, de las más fiables (proteínas curadas) a las más generales (familias de dominios). Anota un genoma bacteriano en borrador en unos diez minutos en un ordenador de escritorio y produce archivos que cumplen los estándares de los repositorios públicos.

Bakta (Schwengers et al., 2021) persigue los mismos objetivos con otras prioridades: ser independiente del taxón, detectar proteínas pequeñas que los predictores suelen pasar por alto y asignar referencias cruzadas precisas a bases de datos públicas. Para ello identifica muchas proteínas *sin alineamiento*, reconociendo secuencias idénticas a las ya catalogadas, lo que acelera el proceso. Exporta GFF3, archivos planos compatibles con INSDC y JSON, y en sus comparaciones superó a otras herramientas rápidas en anotaciones funcionales y referencias cruzadas con tiempos de ejecución comparables.

Anotación de un genoma bacteriano

```
prokka --outdir prokka_cepa --prefix cepa --kingdom Bacteria \
--cpus 8 contigs_500.fasta

bakta --db bakta_db/ --output bakta_cepa --prefix cepa \
--threads 8 contigs_500.fasta

awk '$3 == "CDS"' bakta_cepa/cepa.gff3 | wc -l # nº de CDS
```

⚠ Anotaciones que se copian a sí mismas

La anotación funcional por homología es transitiva: si una proteína se anotó mal en una base de datos, todas las que se anoten por parecido con ella heredarán el error, y la etiqueta equivocada se multiplica con cada genoma nuevo. Desconfíe de funciones muy específicas asignadas con identidades

bajas, prefiera bases de datos curadas y recuerde que “proteína hipotética” es una respuesta honesta. Tampoco compare el número de genes entre genomas anotados con flujos distintos: parte de la diferencia será metodológica, no biológica.

Los genes se encuentran porque “suenan” distinto: ORFs largos que el azar no produce y composición de codones característica, capturada por cadenas de Márkov de fase dentro de un HMM. En eucariotas, enmascarar repeticiones y añadir evidencia de RNA-seq y proteínas es tan importante como el modelo.

☰ En resumen

- Una lectura de longitud L contiene $L-k+1$ k -mers; se cuentan en forma canónica ($\min\{x, \bar{x}\}$) con k impar. La cobertura efectiva de k -mers es $\lambda_k = c \frac{L-k+1}{L} (1-e)^k$.
- El espectro de k -mers $h(m)$ muestra un pico de errores en m pequeños, un pico heterocigoto en λ y uno homocigoto en 2λ . El tamaño del genoma es $\hat{G} = \sum_{m \geq v} m h(m) / 2\lambda$, y la heterocigosidad se obtiene de $\alpha = 1 - (1-h)^k$ (GenomeScope).
- El ensamblaje OLC busca un camino hamiltoniano en el grafo de solapamiento; el de De Bruijn, un camino euleriano en un grafo cuyos nodos son $(k-1)$ -mers y cuyas aristas son k -mers, resoluble en tiempo lineal. Las repeticiones de longitud $\geq k-1$ hacen que el camino no sea único; los ensambladores entregan los unitigs.
- Los errores crean puntas y burbujas (unos NLe^k k -mers espurios), la heterocigosidad crea burbujas equilibradas y las repeticiones, marañas. Solo la información que atraviesa una repetición (pares, lecturas largas) la resuelve.
- SPAdes combina varios k para conectar regiones poco cubiertas y resolver repeticiones; Canu, Flye y hifiasm ensamblan lecturas largas y llevaron al primer genoma humano completo (T2T-CHM13).
- Un ensamblaje se evalúa en contigüidad (N50, NG50, auN), exactitud (QUAST, NGA50, QV de Merquy) y completitud (BUSCO, k -mers). El N50 premia las uniones erróneas.

- La anotación enmascara repeticiones, predice genes con modelos de Márkov y HMM autoentrenados (Glimmer, GeneMark, Prodigal, AUGUSTUS), integra evidencia (BRAKER) y asigna funciones por homología; Prokka y Bakta automatizan el proceso en procariotas.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 8.1** ●●● Escriba todos los 3-mers canónicos de **ACGTTGCA** y su espectro. ¿Hay algún 3-mer que sea su propio reverso complementario? ¿Y algún 4-mer? Relacione la respuesta con la paridad de k .
- 8.2** ●●● Un organismo haploide se secuenció con lecturas de 150 pb. El espectro de 25-mers tiene su pico en $m = 42$, el valle en $m = 6$ y $\sum_{m \geq 6} m h(m) = 1,68 \times 10^9$. Estime el tamaño del genoma y la cobertura por bases c (suponga errores despreciables).
- 8.3** ●●● Construya a mano el grafo de De Bruijn con $k = 3$ de **TTACGTACGC**, compruebe la condición de la ecuación (8.9) e identifique los unitigs. ¿Es único el camino euleriano?
- 8.4** ●●● Con la ecuación (8.4), calcule qué fracción de 21-mers y de 31-mers es heterocigota en un genoma con $h = 0,1\%$ y con $h = 2\%$. ¿En cuál de los cuatro casos el pico heterocigoto supera en altura al homocigoto? Use el resultado para explicar la advertencia de la sección 8.1 sobre confundir picos.
- 8.5** ●●● Modifique `camino_euleriano` para que devuelva *todos* los caminos eulerianos de un grafo pequeño. Aplíquelo al ejemplo de la figura 8.5 con $k = 4$ y $k = 5$ y a un genoma de 30 pb con una repetición de 6 pb: ¿a partir de qué k se vuelve único?
- 8.6** ●●● Descargue lecturas Illumina de un aislado de *E. coli*, ensámblelas con SPAdes usando solo $k = 21$, solo $k = 77$ y la combinación 21, 33, 55, 77. Compare los tres ensamblajes con QUAST frente a K-12 MG1655 (N50, NGA50, errores de ensamblaje, fracción del genoma) y explique las diferencias con la figura 8.7.

- 8.7    Implemente el cálculo de la curva Nx completa y del auN, y demuestre que el auN es igual al área bajo la curva Nx dividida por 100.
- 8.8    Demuestre la ecuación (8.6) incluyendo repeticiones: si una fracción ρ del genoma está en dos copias idénticas, ¿cómo cambia el espectro y por qué el estimador $\hat{G}$ sigue siendo insesgado? ¿Qué ocurre si las dos copias difieren en un 2 %?
- 8.9    Entrene un modelo de Márkov de fase de orden 2 (ecuación (8.18)) con los ORFs de más de 300 codones de *E. coli* y compárelo con el modelo de codones de la ecuación (8.17) como clasificador de los ORFs de 60 a 149 codones, usando como verdad la anotación de RefSeq. Dibuje las curvas de precisión-sensibilidad de ambos.
- 8.10    Programe el algoritmo de Viterbi para un HMM de dos estados (codificante con Márkov de fase de orden 0, no codificante) y úselo para segmentar un fragmento de 20 kb del genoma de *E. coli*. ¿Qué limitación del HMM simple aparece al comparar las longitudes predichas con las de los genes reales?

Referencias del capítulo

- Bankevich, A., Nurk, S., Antipov, D., Gurevich, A. A., Dvorkin, M., Kulikov, A. S., et al. (2012). SPAdes: A new genome assembly algorithm and its applications to single-cell sequencing. *Journal of Computational Biology*, 19(5), 455-477. <https://doi.org/10.1089/cmb.2012.0021>
- Brůna, T., Hoff, K. J., Lomsadze, A., Stanke, M., & Borodovsky, M. (2021). BRAKER2: automatic eukaryotic genome annotation with GeneMark-EP+ and AUGUSTUS supported by a protein database. *NAR Genomics and Bioinformatics*, 3(1), lqaa108. <https://doi.org/10.1093/nargab/lqaa108>
- Cheng, H., Concepcion, G. T., Feng, X., Zhang, H., & Li, H. (2021). Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 18(2), 170-175. <https://doi.org/10.1038/s41592-020-01056-5>
- Compeau, P. E. C., Pevzner, P. A., & Tesler, G. (2011). How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29(11), 987-991. <https://doi.org/10.1038/nbt.2023>
- Delcher, A. L., Bratke, K. A., Powers, E. C., & Salzberg, S. L. (2007). Identifying bacterial genes and endosymbiont DNA with Glimmer. *Bioinformatics*, 23(6), 673-679. <https://doi.org/10.1093/bioinformatics/btm009>
- Gurevich, A., Saveliev, V., Vyahhi, N., & Tesler, G. (2013). QUAST: quality assessment tool for genome assemblies. *Bioinformatics*, 29(8), 1072-1075. <https://doi.org/10.1093/bioinformatics/btt086>

- Hyatt, D., Chen, G.-L., LoCascio, P. F., Land, M. L., Larimer, F. W., & Hauser, L. J. (2010). Prodigal: prokaryotic gene recognition and translation initiation site identification. *BMC Bioinformatics*, 11(1), 119. <https://doi.org/10.1186/1471-2105-11-119>
- Idury, R. M., & Waterman, M. S. (1995). A new algorithm for DNA sequence assembly. *Journal of Computational Biology*, 2(2), 291-306. <https://doi.org/10.1089/cmb.1995.2.291>
- Kokot, M., Długosz, M., & Deorowicz, S. (2017). KMC 3: counting and manipulating k-mer statistics. *Bioinformatics*, 33(17), 2759-2761. <https://doi.org/10.1093/bioinformatics/btx304>
- Kolmogorov, M., Yuan, J., Lin, Y., & Pevzner, P. A. (2019). Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 37(5), 540-546. <https://doi.org/10.1038/s41587-019-0072-8>
- Koren, S., Walenz, B. P., Berlin, K., Miller, J. R., Bergman, N. H., & Phillippy, A. M. (2017). Canu: scalable and accurate long-read assembly via adaptive k-mer weighting and repeat separation. *Genome Research*, 27(5), 722-736. <https://doi.org/10.1101/gr.215087.116>
- Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics*, 2(3), 231-239. [https://doi.org/10.1016/0888-7543\(88\)90007-9](https://doi.org/10.1016/0888-7543(88)90007-9)
- Lukashin, A. V., & Borodovsky, M. (1998). GeneMark.hmm: new solutions for gene finding. *Nucleic Acids Research*, 26(4), 1107-1115. <https://doi.org/10.1093/nar/26.4.1107>
- Manni, M., Berkeley, M. R., Seppey, M., Simão, F. A., & Zdobnov, E. M. (2021). BUSCO update: Novel and streamlined workflows along with broader and deeper phylogenetic coverage for scoring of eukaryotic, prokaryotic, and viral genomes. *Molecular Biology and Evolution*, 38(10), 4647-4654. <https://doi.org/10.1093/molbev/msab199>
- Marçais, G., & Kingsford, C. (2011). A fast, lock-free approach for efficient parallel counting of occurrences of k-mers. *Bioinformatics*, 27(6), 764-770. <https://doi.org/10.1093/bioinformatics/btr011>
- Myers, E. W. (2005). The fragment assembly string graph. *Bioinformatics*, 21(suppl_2), ii79-ii85. <https://doi.org/10.1093/bioinformatics/bti1114>
- Nagarajan, N., & Pop, M. (2013). Sequence assembly demystified. *Nature Reviews Genetics*, 14(3), 157-167. <https://doi.org/10.1038/nrg3367>
- Nurk, S., Koren, S., Rhie, A., Rautiainen, M., Bzikadze, A. V., Mikheenko, A., et al. (2022). The complete sequence of a human genome. *Science*, 376(6588), 44-53. <https://doi.org/10.1126/science.abj6987>
- Pevzner, P. A., Tang, H., & Waterman, M. S. (2001). An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 98(17), 9748-9753. <https://doi.org/10.1073/pnas.171285098>
- Ranallo-Benavidez, T. R., Jaron, K. S., & Schatz, M. C. (2020). GenomeScope 2.0 and Smudgplot for reference-free profiling of polyploid genomes. *Nature Communications*, 11(1), 1432. <https://doi.org/10.1038/s41467-020-14998-3>

- Rhie, A., Walenz, B. P., Koren, S., & Phillippy, A. M. (2020). Merquy: reference-free quality, completeness, and phasing assessment for genome assemblies. *Genome Biology*, 21(1), 245. <https://doi.org/10.1186/s13059-020-02134-9>
- Schwengers, O., Jelonek, L., Dieckmann, M. A., Beyvers, S., Blom, J., & Goesmann, A. (2021). Bakta: rapid and standardized annotation of bacterial genomes via alignment-free sequence identification. *Microbial Genomics*, 7(11), 000685. <https://doi.org/10.1099/mgen.0.000685>
- Seemann, T. (2014). Prokka: rapid prokaryotic genome annotation. *Bioinformatics*, 30(14), 2068-2069. <https://doi.org/10.1093/bioinformatics/btu153>
- Simão, F. A., Waterhouse, R. M., Ioannidis, P., Kriventseva, E. V., & Zdobnov, E. M. (2015). BUSCO: assessing genome assembly and annotation completeness with single-copy orthologs. *Bioinformatics*, 31(19), 3210-3212. <https://doi.org/10.1093/bioinformatics/btv351>
- Stanke, M., & Waack, S. (2003). Gene prediction with a hidden Markov model and a new intron submodel. *Bioinformatics*, 19(suppl_2), ii215-ii225. <https://doi.org/10.1093/bioinformatics/btg1080>
- Tarailo-Graovac, M., & Chen, N. (2009). Using RepeatMasker to identify repetitive elements in genomic sequences. *Current Protocols in Bioinformatics*, 25(1). <https://doi.org/10.1002/0471250953.bi0410s25>
- Vurture, G. W., Sedlazeck, F. J., Nattestad, M., Underwood, C. J., Fang, H., Gurtowski, J., & Schatz, M. C. (2017). GenomeScope: fast reference-free genome profiling from short reads. *Bioinformatics*, 33(14), 2202-2204. <https://doi.org/10.1093/bioinformatics/btx153>
- Yandell, M., & Ence, D. (2012). A beginner's guide to eukaryotic genome annotation. *Nature Reviews Genetics*, 13(5), 329-342. <https://doi.org/10.1038/nrg3174>
- Zerbino, D. R., & Birney, E. (2008). Velvet: Algorithms for de novo short read assembly using de Bruijn graphs. *Genome Research*, 18(5), 821-829. <https://doi.org/10.1101/gr.074492.107>